Andrii Hurskykh

Detecting Suspicious Logins in Windows and Linux Using Wazuh

Bachelor’s Thesis

Metropolia University of Applied Sciences
Bachelor of Engineering
Information Technology
Group: TXL26S2

Table of Contents

Abstract

List of Abbreviations

1 Introduction
1.1 Background and Motivation
1.2 Aim and Research Questions
1.3 Scope of the Thesis
1.4 Structure of the Thesis

2 Authentication Logging and Threat Detection
2.1 Windows Logon Events
2.2 Linux SSH Authentication Logs
2.3 Suspicious Login Patterns
2.4 Wazuh and Its Detection Capabilities

3 Design of the Test Environment
3.1 Laboratory Architecture
3.2 Windows and Linux Endpoints
3.3 Wazuh Installation and Log Collection
3.4 Test Scenarios and Evaluation Criteria

4 Implementation and Testing
4.1 Verification of Log Collection
4.2 Normal Login Activity
4.3 Repeated Failed Login Attempts
4.4 Successful Login Following Failed Attempts
4.5 Testing Detection Rules

5 Results and Discussion
5.1 Windows Detection Results
5.2 Linux Detection Results
5.3 False Positives and Missed Events
5.4 Limitations of the Test Environment

6 Conclusions

References

Appendices
Appendix 1. Laboratory Configuration
Appendix 2. Test Scenarios and Results
Appendix 3. Wazuh Rules and Queries

Abstract

This thesis examines how Wazuh collects and reports authentication activity from Windows and Linux and whether the resulting evidence supports investigation of suspicious login patterns. A controlled laboratory combined a Windows 11 endpoint, an Ubuntu SSH server and an all-in-one Wazuh deployment in VirtualBox. The scenarios covered successful authentication, repeated failed attempts and successful authentication following failures.

The evaluation compared native Windows Security events and Ubuntu SSH journal messages with Wazuh alerts. Windows testing used a dedicated local account and interactive authentication, while Linux testing used SSH password authentication from the Windows host. Account names, event identifiers, timestamps and available source details were used to match the endpoint records to the collected evidence.

Wazuh reported successful and failed authentication on both platforms. Windows events 4624 and 4625 were associated with rules 60118 and 60122. Linux accepted and failed passwords were reported under rules 5715 and 5760, and a PAM summary of additional failures triggered rule 2502 at level 10. The records supported manual reconstruction of a successful login after failed attempts. However, a dedicated alert correlating the complete sequence was not demonstrated.

The results establish event visibility and rule classification for the tested scenarios. They also show why individual authentication alerts require contextual interpretation. The limited environment, small number of trials and unresolved timestamp differences prevented reliable estimates of detection rates, false-positive rates or ingestion latency. Further evaluation should verify clock synchronisation, retain structured exports and test a defined correlation rule against repeated normal and suspicious sequences.

Keywords: Wazuh, authentication logging, Windows Security, Linux, SSH, login monitoring

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ATT&CK	Adversarial Tactics, Techniques, and Common Knowledge
ID	Identifier
IP	Internet Protocol
JSON	JavaScript Object Notation
LTS	Long Term Support
NTLM	NT LAN Manager
PAM	Pluggable Authentication Modules
RAM	Random Access Memory
RDP	Remote Desktop Protocol
SID	Security Identifier
SSH	Secure Shell
UTC	Coordinated Universal Time
VM	Virtual Machine
WQL	Wazuh Query Language
XML	Extensible Markup Language

\

1 Introduction

1.1 Background and Motivation

Authentication logs provide information about attempts to access computer systems. In Windows, security event 4624 records a successful logon, while event 4625 records a failed logon attempt (Microsoft, n.d.-a; Microsoft, n.d.-b). Linux systems can also record authentication activity generated by the Secure Shell (SSH) service. These records give an analyst a starting point for investigating unusual login behaviour.

A failed login does not necessarily indicate an attack: a legitimate user may enter an incorrect password. However, repeated failures, especially when followed by a successful login, may warrant closer examination. MITRE ATT&CK describes brute force as a technique in which an adversary repeatedly attempts to obtain access to an account (MITRE, n.d.). The challenge for a monitoring system is therefore to identify useful signals without treating every routine mistake as an incident.

Wazuh is an open-source security platform that can collect and analyse log data from Windows and Linux endpoints. Its documentation includes an example of detecting repeated authentication failures against Windows and Linux systems (Wazuh, n.d.-a; Wazuh, n.d.-b). This makes it suitable for a small laboratory project in which the original authentication events can be compared with the alerts generated by a monitoring platform.

The motivation for this thesis is to examine that comparison in practice. Rather than assuming that the presence of a log entry guarantees a useful alert, the thesis will test what information is collected, which activities generate alerts and what limitations become visible in a controlled environment.

1.2 Aim and Research Questions

The aim of this thesis is to build a small Windows and Linux authentication monitoring environment using Wazuh and evaluate its ability to identify selected suspicious login patterns. The work will produce a documented laboratory setup, repeatable test scenarios and an assessment of the resulting events and alerts.

The thesis addresses the following research questions:

1. Which authentication events from the selected Windows and Linux systems can be collected and examined in Wazuh?

2. How does the configured Wazuh environment respond to normal logins, repeated failed logins and a successful login following failed attempts?
3. What limitations and potential false positives are observed in the tested scenarios?

The answers will be based on events generated during the tests and on a comparison between the original system logs and the information available in Wazuh.

1.3 Scope of the Thesis

The practical work will be conducted in an isolated laboratory using one Windows endpoint, one Linux endpoint and a Wazuh installation. The investigation will focus on Windows logon events and Linux SSH authentication events. The planned tests will cover normal login activity, repeated failed attempts and a successful login after failures.

The thesis will evaluate only the configurations, software versions and scenarios documented in the laboratory. It will not assess an organisation's production environment, establish that an account was actually compromised or attempt to measure Wazuh's effectiveness against every possible authentication attack. Automated blocking and incident response are outside the planned scope.

1.4 Structure of the Thesis

Chapter 2 explains the authentication events and detection concepts used in the work. Chapter 3 describes the laboratory design, log sources, test scenarios and evaluation criteria. Chapter 4 documents the implementation and testing process. Chapter 5 presents and discusses the observed results, including limitations and false positives. Chapter 6 summarises the conclusions and proposes possible improvements.

2 Authentication Logging and Threat Detection

This chapter establishes the technical basis for detecting suspicious authentication activity in Windows and Linux. It describes the evidence produced by the operating systems, the behaviour that can make a sequence of logins worth investigating, and

the way Wazuh converts collected events into alerts. The distinction between an operating-system event, a Wazuh alert and a confirmed security incident is maintained throughout the chapter. The configuration and observed results of the thesis laboratory are presented in Chapters 3–5.

2.1 Windows Logon Events

Windows records security-relevant authentication activity in the Security event log when the corresponding audit settings are enabled. For this thesis, the two central records are Event ID 4624, which reports that a logon session was created, and Event ID 4625, which reports an unsuccessful logon attempt. Microsoft notes that a successful logon event is produced on the destination computer where the session is created. A failed logon is likewise recorded on the computer where the attempt was made. Consequently, searching only the workstation from which a connection was initiated can miss the decisive event on the target host (Microsoft, 2021; Microsoft, 2026).

These event IDs describe outcomes, not motives. Event 4624 can follow an ordinary interactive session, a scheduled task, a service start or remote access. Event 4625 can reflect a typing error, an outdated stored password, a disconnected service or deliberate credential guessing. The analyst must examine the event fields and the surrounding activity before interpreting either event as suspicious. This distinction is particularly important in a small laboratory: a test can deliberately produce an alert without representing a real intrusion (Microsoft, 2021; Microsoft, 2026).

The logon type identifies how access was requested. Type 2 refers to an interactive local logon, type 3 to a network logon and type 10 to a remote interactive logon, such as a Remote Desktop session. Types 4 and 5 represent batch and service logons. These categories help distinguish a person signing in remotely from a background process that creates its own session. They also prevent a misleading comparison of unlike events: a type 3 failure for an account and a type 2 success for the same account might have separate explanations. Logon type is therefore a useful filter, though it does not independently establish whether an event is benign or malicious (Microsoft, 2021; Microsoft, n.d.-a).

Event or field	Meaning	Use in this thesis
4624	A logon session was successfully created.	Inspect the account, destination host, logon type and available

Event or field	Meaning	Use in this thesis
		source details; compare with preceding failures.
4625	A logon attempt failed.	Count attempts and examine the target account, logon type, source and failure reason.
Logon type 2 / 3 / 10	Interactive / network / remote interactive access.	Separate expected local activity from the remote-access scenario being tested.
Status and substatus	Additional information about a failed attempt.	Distinguish causes of failure when the fields are present and meaningful.

Table 1. Principal Windows authentication events and fields relevant to the planned tests (adapted from Microsoft, 2021; Microsoft, 2026; Microsoft, n.d.-a).

An event contains more than an ID. In 4625, the “Account For Which Logon Failed” section identifies the account targeted by the attempt, whereas the “Subject” section describes the security context that requested the operation. Confusing the two can lead to attributing an action to the wrong account. The failure information can also provide a status and substatus, which help explain why access was rejected. The workstation name, source network address and source port provide possible network context, but their availability depends on the authentication process and the circumstances of the event; an absent value should not automatically be read as evidence of concealment (Microsoft, 2026).

The corresponding 4624 record has a “New Logon” section for the account receiving the session, along with a logon type and, where available, network information. The logon ID can assist in linking events that concern a particular session on the same computer. For a comparison across separate failed and successful attempts, however, the investigation normally relies on a combination of host, account, time, logon type and available source information. It would be unsound to assume that two independent attempts must have the same logon ID or that a source address will always be populated (Microsoft, 2021; Microsoft, 2026).

Audit policy determines whether the required events are actually available. Microsoft places these records in its Audit Logon category, with success and failure auditing configurable separately. Before a detection test, the investigator should confirm both that Windows generated the expected event locally and that Wazuh collected it. An empty Wazuh search is not by itself proof that no login was attempted: the audit policy, event channel, agent configuration, communication path and search filter all require examination. The laboratory implementation will document the relevant settings instead of assuming that every Windows installation produces identical evidence (Microsoft, n.d.-b; Wazuh, n.d.-a).

Account lockout is a related but distinct event. Multiple failures might lock an account, and the resulting behaviour can alter what later attempts produce. The present thesis centres on 4624 and 4625 and treats lockout information as context if it appears. Likewise, a remote Windows test should specify the access method actually used. Calling every 4625 event an “RDP failure” would be inaccurate if its logon type or circumstances indicate a different service. This matters when the same event IDs are used to analyse both an ordinary sign-in and a remote-access sequence (Microsoft, 2026; Microsoft, n.d.-a).

The practical value of Windows authentication logging lies in reconstructing a sequence. A single record can state that an attempt succeeded or failed; a sequence can show whether the same account was tried repeatedly, whether several accounts were targeted from one source, and whether a successful session followed the failures. Such conclusions remain conditional on the completeness and accuracy of the available fields. In Chapter 4, the raw Windows records will therefore be compared with any Wazuh alerts rather than treating the alert count as a complete record of login attempts.

2.2 Linux SSH Authentication Logs

In the Linux part of this thesis, the monitored access method is Secure Shell (SSH). An SSH server accepts a connection and applies its configured authentication methods before granting access. OpenSSH supports more than one method, including public-key and password authentication, and its logging behaviour can be influenced by server configuration. The precise messages generated in a test consequently depend on the enabled method, the SSH daemon and the surrounding system configuration. The thesis will record those settings before interpreting failed and successful attempts (Ubuntu, n.d.-a).

Linux authentication evidence may be exposed as text-based system logs, as entries in the systemd journal, or through both mechanisms. Wazuh documentation provides `/var/log/auth.log` as an example file source and also describes direct collection from journald. The location should be verified on the actual Linux endpoint: a configuration that points to a nonexistent file cannot be assumed to collect SSH attempts merely because that path is familiar from another distribution. A system using journald can be inspected by service or other journal fields, while a file-based configuration can be checked against the actual authentication log on disk (Wazuh, n.d.-a; Wazuh, n.d.-b; systemd, n.d.).

A typical SSH authentication message contains a timestamp, host or service identifier, outcome, account name and sometimes the origin IP address and source port. A rejected password for an existing account, an attempt directed at an invalid account and an accepted password or public key represent different observations. The exact wording is a property of the operating system and software combination, rather than a universal Linux event ID. Unlike Windows Event ID 4625, an SSH failure must normally be recognised through the structure and content of a message or through fields extracted from that message (Wazuh, n.d.-c; Ubuntu, n.d.-a).

The distinction between a failed password and an invalid user is analytically useful. An attacker may test common account names before attempting passwords, while a legitimate user may mistype a valid account name. Repetition, origin and timing determine whether the pattern deserves attention. Successful SSH authentication is equally contextual: it can be an authorised administrator returning to the host, or it can complete an earlier guessing sequence. An “Accepted” message alone does not reveal who physically controlled the remote computer, and the log cannot prove that the account holder intended the connection.

SSH authentication methods also affect the proposed experiment. If password authentication is disabled, repeated incorrect passwords will not produce the intended password-failure sequence against that server. Conversely, enabling password access solely to make the laboratory scenario possible should be documented as a deliberate test setting, not presented as a general security recommendation. For key-based authentication, the presence of a successful login can be examined without implying that password guessing was possible. The implementation chapter will specify which method was enabled for each scenario and will not generalise those results to all SSH installations (Ubuntu, n.d.-a).

For a traceable test, the event should first be found on the Linux endpoint, then in the data collected by Wazuh, and finally in the relevant Wazuh alert or search result. If the first step succeeds but the second fails, collection is the likely area to investigate. If collection succeeds but an expected alert is missing, the next checks are decoding, rule conditions, alert threshold and search filters. These are diagnostic possibilities rather than findings about the thesis environment; the actual cause can only be established from the configuration and test evidence (Wazuh, n.d.-c; Wazuh, n.d.-d).

Linux and Windows thus expose comparable security questions through different evidence formats. Both can show failed and successful attempts and may supply account, time and origin. Windows provides structured event IDs and logon types, whereas SSH commonly produces service messages that require parsing. The evaluation should compare whether each platform's relevant activity was observable and detected, while preserving the platform-specific fields needed to explain the result.

2.3 Suspicious Login Patterns

A suspicious login is an interpretation of events in context, rather than an operating-system event category. The same failed attempt can be routine on one system and unusual on another. Time, frequency, identity, source, access method and what happens next all affect the assessment. MITRE ATT&CK describes brute force as an adversary technique for attempting to obtain credentials through repeated guesses, with related variations such as password spraying. This framework provides vocabulary for analysing patterns; assigning an ATT&CK label does not prove that a particular alert was caused by an adversary (MITRE ATT&CK, n.d.).

The simplest pattern is repeated unsuccessful authentication against the same account. A short burst may suggest password guessing, but it can also be produced by a user forgetting a password, a stale credential saved in an application, or a deliberately executed thesis test. To assess the pattern, the analyst examines whether the attempts share a source, how closely they occur, whether the logon type or service remains the same, and whether the target account is a person, a service or an administrator. A detection rule that counts failures without this context may be useful as an initial alert but will often need review.

A second pattern involves one source attempting several accounts. A small number of tries against each username can escape a simple per-account threshold while still producing a noteworthy source-based sequence. Password spraying is a related

credential-guessing pattern that uses a small set of passwords against many accounts; the logs in this thesis can reveal the distribution of failed attempts across usernames, but generally cannot establish which candidate password was supplied in each attempt. The distinction between observed behaviour and inferred password strategy should therefore remain explicit (MITRE ATT&CK, n.d.).

A third pattern is a successful login soon after several failures. This sequence can be concerning because a guessed password might eventually have worked. It is not sufficient evidence of a compromised account: a legitimate user may remember the correct password after a few attempts. Stronger contextual support comes from a consistent account and source, a short interval, an unexpected time or host, and a remote-access method matching the failed attempts. Even if these conditions are met, the correct wording is “a suspicious sequence requiring investigation”, unless separate evidence establishes unauthorised access.

Pattern	Evidence to correlate	Possible benign explanation	Interpretation limit
Several failures for one account	Account, source, host, time window, access method	Incorrect saved password or user error	Failures alone do not show compromise.
One source targets several accounts	Source, distinct usernames, time window	Administrative testing or misconfigured automation	The password values normally are not visible.
Success follows failures	Account, host, source, access method, event order	Legitimate user eventually enters correct credentials	A success does not establish attacker control.
One isolated failure	Outcome and local context	Routine typing mistake	Too weak to classify as an attack.

Table 2. Working interpretations for the controlled scenarios in this thesis. The explanations are analytical possibilities, not results of testing.

The choice of a time window and a repetition threshold affects what an alert detects. A low threshold may create frequent alerts during ordinary use; a high threshold may fail to report shorter guessing sequences. A very short window can split a longer sequence into separate groups, while a very long window can join unrelated user errors. For the laboratory, the threshold and window should be stated before running tests, the number of attempts should be recorded, and the observed Wazuh output should be compared against that record. This permits a clear discussion of both missed activity and extra alerts without claiming a universal best threshold.

Correlation across platforms requires further caution. A Windows workstation and a Linux server might record different identifiers for what appears to be the same account, and network address translation or a shared jump host may make unrelated users appear under the same source address. Clock differences can also change the apparent order of events. The experiment should keep a consistent time reference and identify each endpoint in the test record. In this thesis, the Windows and Linux scenarios are evaluated separately first; any cross-platform interpretation will be limited to what the recorded fields can support.

The terms “event”, “alert”, “detection” and “incident” have separate meanings in the evaluation. An event is a logged observation, such as a failed SSH password or a Windows 4625 record. An alert is the output of a rule that matched one or more observations. A detection is a correct recognition of an activity selected in the test plan. An incident would require an additional judgment that an actual security problem occurred. Because the scenarios are controlled and authorised, their alerts demonstrate detection behaviour, not real compromise. The false-positive discussion in Chapter 5 will use the test plan’s expected outcomes as its comparison point.

2.4 Wazuh and Its Detection Capabilities

Wazuh supplies a common analysis layer for logs produced by different operating systems. Its architecture comprises agents on monitored endpoints and central components consisting of a Wazuh server, indexer and dashboard. The agent collects the selected data; the server analyses incoming events; the indexer stores searchable alert data; and the dashboard provides a way to examine results. This division matters when troubleshooting: a missing dashboard alert might originate from the endpoint, the collection settings, transport, decoding, a rule condition or the search used to display the result (Wazuh, n.d.-e; Wazuh, n.d.-f).

The collection settings are platform specific. A Windows agent can read the Security event channel through an `eventchannel` configuration. A Linux agent can monitor an authentication log file, and Wazuh also supports reading from journald. The correct source is whichever one contains the relevant SSH messages in the chosen laboratory installation. Specifying the source explicitly makes the experiment reproducible: the reader can see where the original data came from rather than assuming that an installed agent automatically captures all possible logs (Wazuh, n.d.-a; Wazuh, n.d.-b).

After collection, Wazuh decoders parse messages into fields that its rules can evaluate. The rule set contains built-in detection logic and can be extended with custom rules. Matching a basic rule may produce an alert for one failed authentication, whereas correlation rules can trigger when relevant events recur within a defined period. Wazuh describes its processing in stages that include decoding and rule matching; its test tool, `wazuh-logtest`, permits a sample message to be checked against the applicable decoder and rules. The rule identifier and alert level help explain why a given event appeared in the interface, but their meaning must be checked against the installed ruleset (Wazuh, n.d.-c; Wazuh, n.d.-d; Wazuh, n.d.-g).

Wazuh documentation includes a proof of concept for detecting repeated SSH and Windows Remote Desktop authentication failures. It lists SSH-related alert rules and Windows-related rules to inspect, while its general log-analysis examples also show an SSH authentication-success rule. These examples establish that built-in detection exists for relevant event types. They do not guarantee that the same identifiers or alert behaviour will appear in every thesis environment: the collected source, installed version, scenario and rules in effect must be verified in Chapter 4 (Wazuh, n.d.-h; Wazuh, n.d.-c).

A useful property of Wazuh for this thesis is its ability to place different operating-system alerts into one searchable environment. The evaluation can look for the endpoint name, event time, account, source address where present, original message and rule description. At the same time, Wazuh’s unified view should not hide differences in the original evidence. A Windows event’s logon type, for example, has no exact equivalent in a typical SSH text message; an SSH “invalid user” message is not itself a Windows event ID. The raw source record will be retained as a reference when interpreting each alert.

Rules may also be tailored to the laboratory. Wazuh supports custom conditions and correlation parameters, including frequency and a time frame, so a narrowly defined repeated-failure rule can be tested if a built-in alert does not match the planned question. The priority is to document the exact rule logic, the expected triggering sequence and the observed behaviour. Any local rule changes will be recorded in Appendix 3 and described separately from built-in rules; otherwise the reader could mistake a result caused by the thesis configuration for a standard product capability (Wazuh, n.d.-g; Wazuh, n.d.-i).

The `wazuh-logtest` utility is useful for checking how a particular sample is decoded and which rule matches it. It does not by itself prove that an agent collected a real operating-system event, that the server received it during a live test or that an analyst could find the resulting alert in the dashboard. For this reason, Chapter 4 will use two complementary checks: rule testing on representative samples where needed, and an end-to-end test that begins with an actual authorised login attempt on the monitored host and ends with comparison of the host log and Wazuh results (Wazuh, n.d.-d; Wazuh, n.d.-f).

An alert’s absence must be interpreted with the same care as its presence. Some events are collected but do not meet the active alert rule; others may be excluded by a search filter or have too little information for the intended correlation. Conversely, a repeated-failure alert can accurately report its rule conditions while describing ordinary user behaviour. The thesis therefore assesses technical detection against known generated scenarios and discusses investigation context separately. This gives the results chapter a defensible basis for reporting detected scenarios, missed scenarios and alerts that could mislead an analyst.

In summary, Windows and Linux produce evidence that can describe authentication outcomes, but each system structures and stores that evidence differently. Wazuh can collect and analyse both sources, and its rules can report individual and repeated authentication events. The design of the following laboratory chapter follows from these observations: it must establish the actual log source on each endpoint, record the authentication method and audit settings, generate known sequences of successful and unsuccessful attempts, and compare original events with Wazuh output. Whether the configured environment achieves those aims remains an empirical question for Chapters 4 and 5.

References used in Chapter 2

Microsoft (2021). 4624(S): An account was successfully logged on. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4624

Microsoft (2026). 4625(F): An account failed to log on. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4625

Microsoft (n.d.-a). Administrative tools and logon types reference. https://learn.microsoft.com/en-us/windows-server/identity/securing-privileged-access/reference-tools-logon-types

Microsoft (n.d.-b). Audit Logon. https://learn.microsoft.com/en-us/windows/security/threat-protection/auditing/audit-logon

MITRE ATT&CK (n.d.). Brute Force, Technique T1110. https://attack.mitre.org/techniques/T1110/

systemd (n.d.). journalctl. https://www.freedesktop.org/software/systemd/man/255/journalctl.html

Ubuntu (n.d.-a). sshd_config manual page. https://manpages.ubuntu.com/manpages/noble/man5/sshd_config.5.html

Wazuh (n.d.-a). Configuring log collection for different operating systems. https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html

Wazuh (n.d.-b). Journald log collection. https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/journald.html

Wazuh (n.d.-c). Log data analysis. https://documentation.wazuh.com/current/getting-started/use-cases/log-analysis.html

Wazuh (n.d.-d). Testing decoders and rules. https://documentation.wazuh.com/current/user-manual/ruleset/testing.html

Wazuh (n.d.-e). Components. https://documentation.wazuh.com/current/getting-started/components/index.html

Wazuh (n.d.-f). How log data collection works. https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/how-it-works.html

Wazuh (n.d.-g). Rules syntax. https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html

Wazuh (n.d.-h). Detecting a brute-force attack. https://documentation.wazuh.com/current/proof-of-concept-guide/detect-brute-force-attack.html

Wazuh (n.d.-i). Custom rules. https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html

3 Design and Implementation of the Test Environment

This chapter describes the controlled laboratory built to examine Windows and Linux authentication events with Wazuh. It records the deployed components, log sources and test procedure. Screenshots provide implementation and event evidence; Chapter 4 analyses the results and the limits of the observed alerts.

3.1 Laboratory Architecture

The laboratory consists of a Windows 11 Pro host, a VirtualBox virtual machine running the Wazuh manager, indexer and dashboard, and a second virtual machine running Ubuntu Server and SSH. One Wazuh agent runs on the Windows host and another on Ubuntu. The Windows host is also the operator workstation from which the dashboard and controlled SSH tests are accessed. The combined Wazuh components follow an all-in-one deployment suitable for a small laboratory (Wazuh, n.d.-a; Wazuh, n.d.-b).

The components have distinct roles. The endpoint operating system first creates its native authentication record. The local Wazuh agent collects the configured event source and forwards relevant data to the Wazuh server. The server decodes and evaluates events against rules; alert data is then available through the indexer and dashboard. The host workstation is used to access the dashboard and to carry out the authorised login tests. By keeping the Windows and Linux systems separate, a missing record can be traced to the relevant endpoint and collection path without confusing one operating system's evidence with the other's (Wazuh, n.d.-b; Wazuh, n.d.-c).

Component	Proposed role	Evidence or output
Windows host	Windows 11 Pro endpoint and operator workstation; agent 001; 192.168.56.1.	Security events 4624/4625; SSH test source.
Wazuh virtual machine	Wazuh server, indexer and dashboard; 192.168.56.101.	Connected agent status and searchable alerts.
Ubuntu virtual machine	Ubuntu 26.04.1 LTS SSH server; agent 002; 192.168.56.102.	SSH journald messages and Wazuh Linux alerts.

Table 3. Implemented laboratory components and their roles.

The VirtualBox host-only network uses 192.168.56.1 for the Windows host, 192.168.56.101 for the Wazuh virtual machine and 192.168.56.102 for the Ubuntu

SSH virtual machine. It carries laboratory traffic without requiring the monitored SSH service to be publicly accessible. During the tests, the Windows host at 192.168.56.1 initiates SSH connections to Ubuntu at 192.168.56.102. The event records are interpreted using the origin address reported by each system.

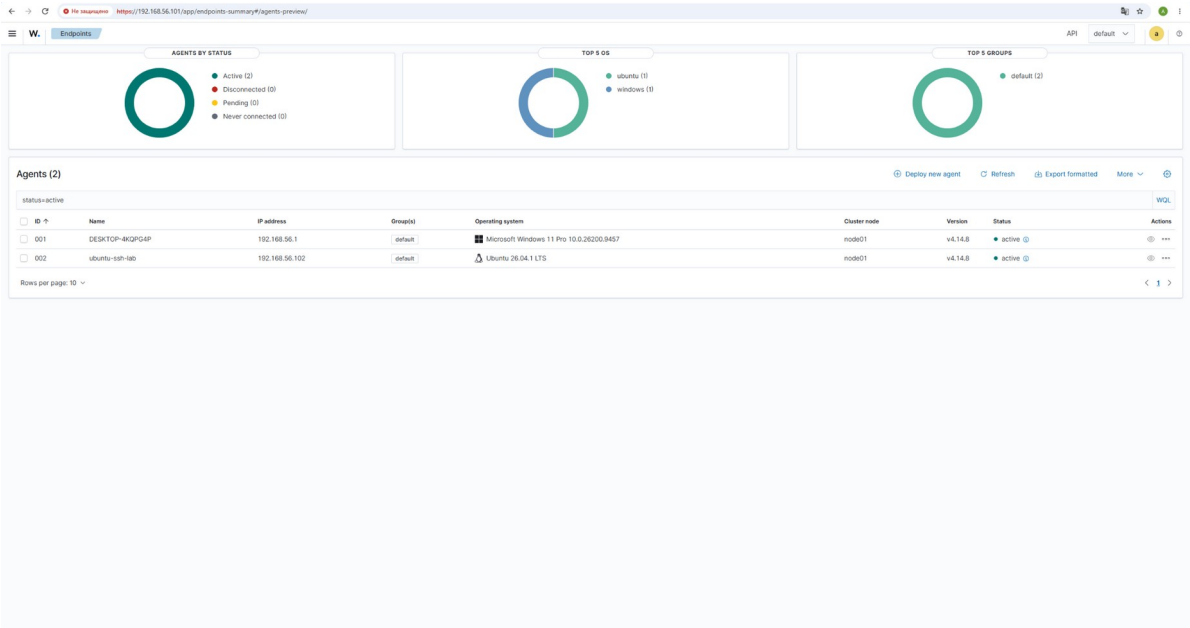


Figure 3.1. Final inventory showing active Windows and Ubuntu Wazuh agents.

The Wazuh virtual machine hosts the server, indexer and dashboard. Wazuh recommends 4 virtual CPUs, 8 GiB of RAM and 50 GB of storage for an all-in-one deployment of 1–25 agents in its quickstart example. These are published planning figures; this chapter does not claim that the virtual machine was allocated those exact resources (Wazuh, n.d.-a).

The host and virtual machines must be compared using a consistent time reference. The screenshots display dashboard times, while native logs and exported timestamps may use a different zone. The order and source details of the failed and successful SSH events are checked directly in the same dashboard view. Any future virtual-machine snapshot restoration should be logged because it may change the apparent sequence of events.

The chosen scope excludes a production network, an Active Directory domain, real user accounts and internet-exposed services. The purpose is to test whether defined authentication sequences can be observed and how Wazuh responds to them. This deliberately limited environment improves control over the test actions, although it

cannot reproduce the variety of users, applications and background activity found in a large organisation.

3.2 Windows and Linux Endpoints

The Windows endpoint is the physical Windows 11 Pro host DESKTOP-4KQPG4P, monitored as Wazuh agent 001 at 192.168.56.1. Its Security event channel supplies authentication evidence. A deliberately invalid local account name, labtest_invalid, was used for the failed-logon test; existing Windows account activity supplied examples of successful logons. Windows 4624 entries must be interpreted by logon type: the type 7 unlock and type 11 cached interactive examples shown below are distinct records, not a single login (Microsoft, 2021a; Microsoft, 2026).

The Security log is the reference source for Windows test events. The endpoint must generate the intended successful and failed records locally before an absence or presence of Wazuh alerts can be interpreted. Event IDs 4624 and 4625 provide the principal outcomes, and the logon type and available account and source fields are used to determine whether a record belongs to a particular test action. Successful and failed logon auditing will be checked on the installed system; it is not sufficient to assume that a default configuration produces every event required by the test plan (Microsoft, 2021a; Microsoft, 2021b; Microsoft, 2026).

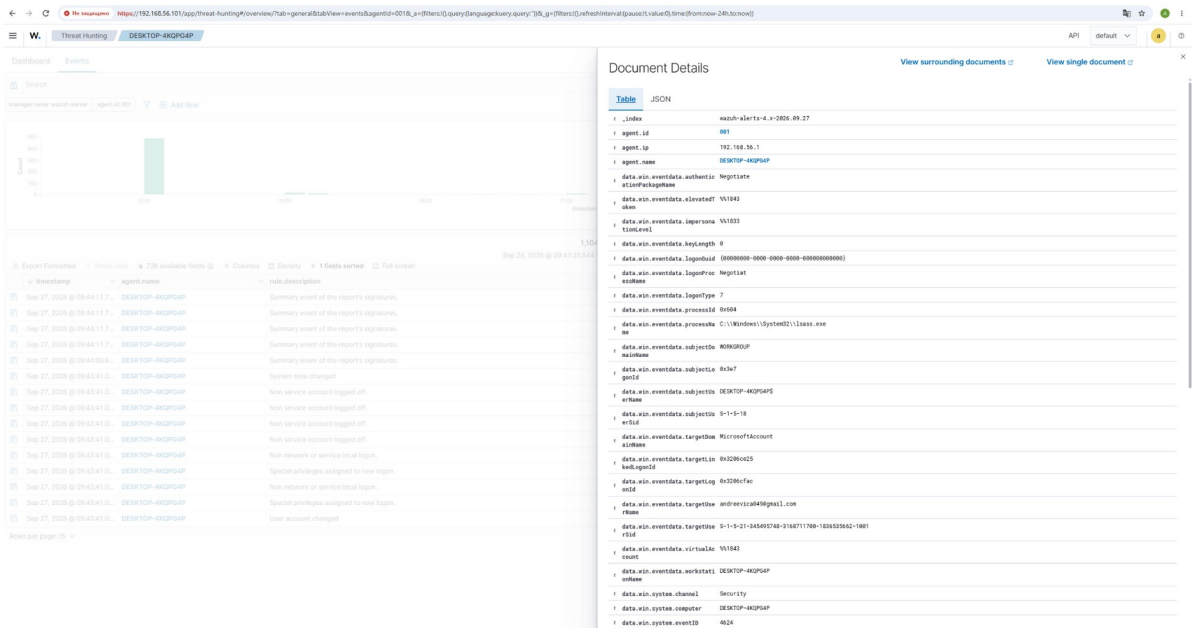


Figure 3.2. Windows 4624 event showing logon type 7 (unlock).

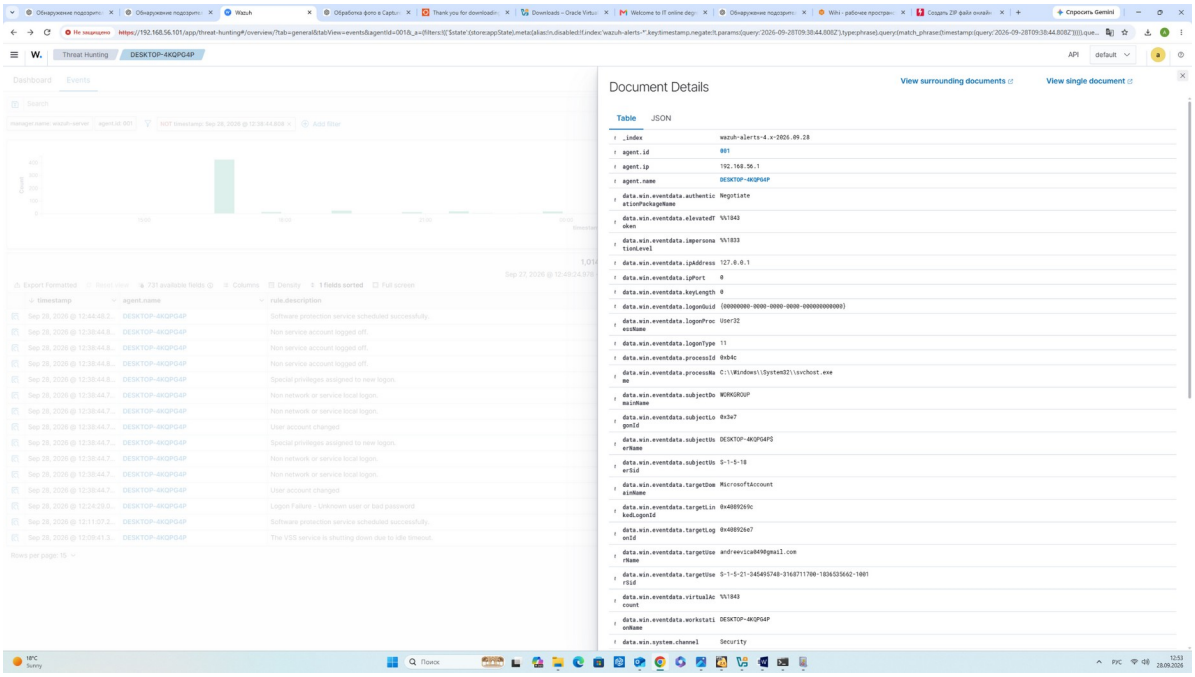


Figure 3.3. Windows 4624 event detail showing logon type 11 (cached interactive).

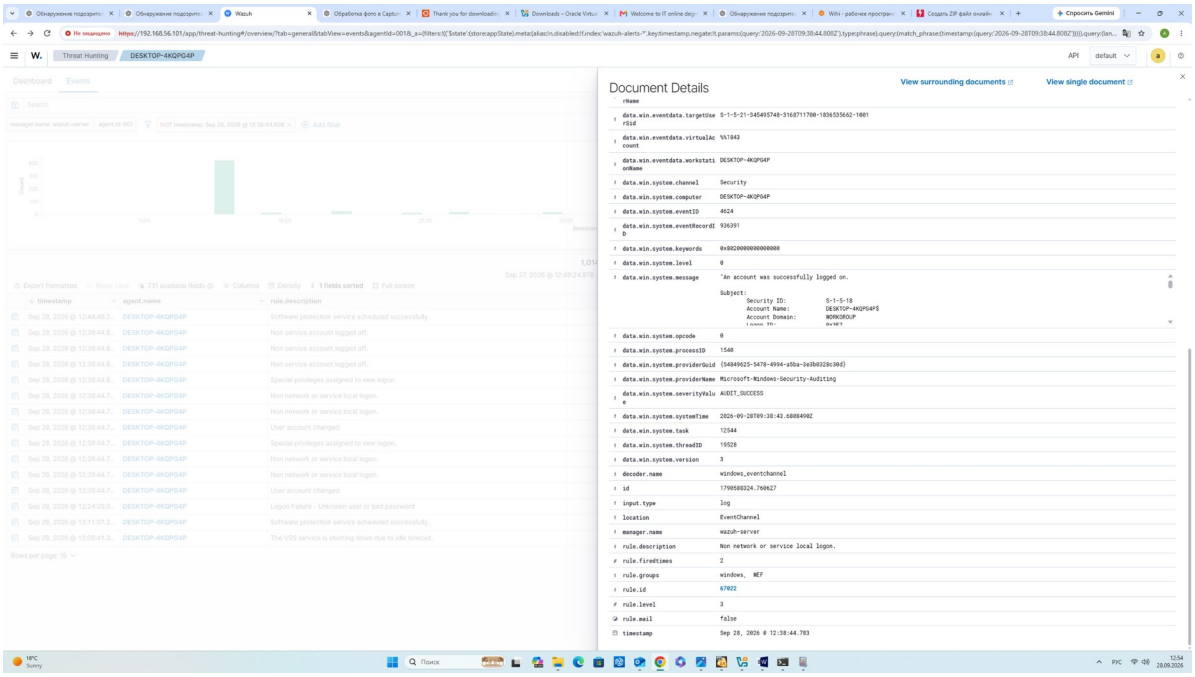


Figure 3.4. Wazuh event detail for the Windows cached logon, rule 67022.

The Linux endpoint is the Ubuntu 26.04.1 LTS virtual machine ubuntu-ssh-lab at 192.168.56.102. It is monitored as Wazuh agent 002. OpenSSH accepts password authentication for the lab account labuser in the isolated host-only network. Controlled failed and successful SSH attempts originate from the Windows host. This test addresses password authentication; an SSH key-based login would constitute a separate experiment (Ubuntu, n.d.).

The Linux SSH service records the attempts in journald, which was checked with journalctl -u ssh; /var/log/auth.log is also present. The Wazuh event details for the demonstrated SSH alerts report location journald. The source is identified from the observed alert fields instead of assuming that every Ubuntu installation uses the same authentication log path (Wazuh, n.d.-d; Wazuh, n.d.-e).

Baseline verification located Windows Security logon events and Ubuntu SSH success and failure messages. The observations were then compared with corresponding Wazuh events for agents 001 and 002. This checks the native source before attributing an absence of alerts to a Wazuh rule. The successful Windows examples shown here are type 7 and type 11 logons, while the controlled Windows failure is a separate type 2 attempt.

The two endpoint configurations should be comparable in purpose while preserving their differences in implementation. Each has one monitored agent and one defined authentication method for its primary scenario. The Windows endpoint is evaluated through structured Security events and their logon types; the Linux endpoint is evaluated through SSH service messages. An equal number of login actions on the two systems will not necessarily produce an equal number of native records or Wazuh alerts. The thesis compares the observed sequence against the documented expectation for each platform, rather than requiring equal alert counts.

3.3 Wazuh Installation and Log Collection

A single Wazuh virtual machine provides the manager, indexer and dashboard at 192.168.56.101. The deployed agents report version 4.14.8 in the dashboard. The dashboard itself establishes interface availability; the agent inventory and the individual alerts shown below establish that both endpoint data streams are visible (Wazuh, n.d.-a; Wazuh, n.d.-b).

The Windows host was registered as DESKTOP-4KQPG4P, agent 001, and Ubuntu was registered as ubuntu-ssh-lab, agent 002. Both are shown as active in the endpoint inventory. The earlier single-agent views document the intermediate installation state, and the two-agent view confirms the final tested configuration (Wazuh, n.d.-f; Wazuh, n.d.-g; Wazuh, n.d.-h).

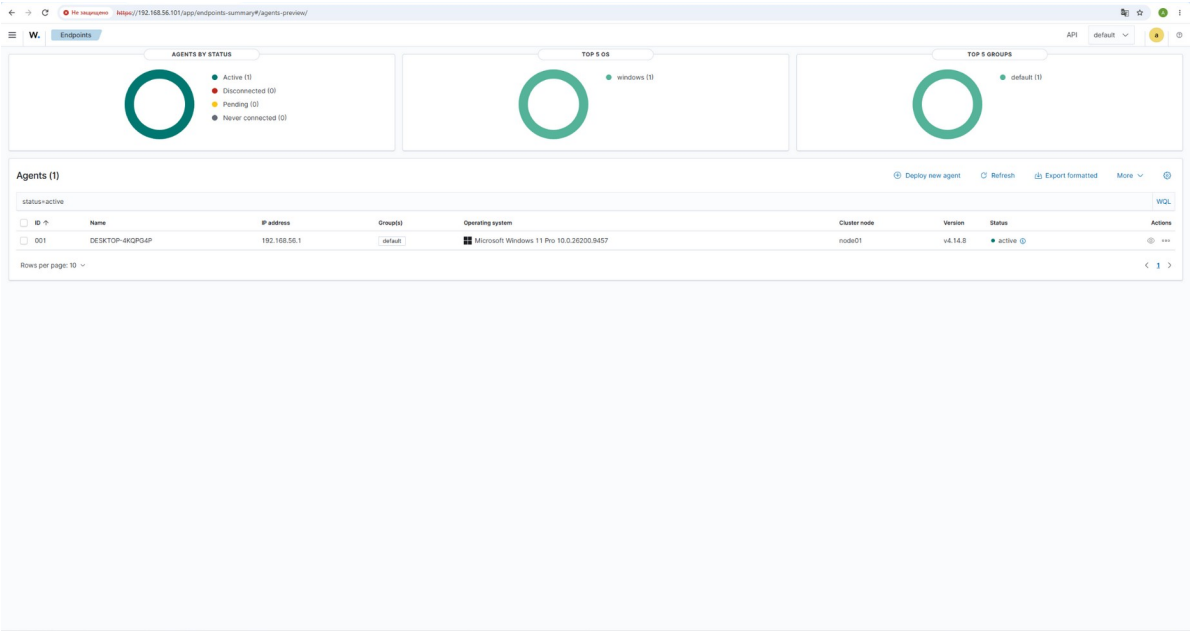


Figure 3.5. Intermediate inventory showing the Windows agent active.

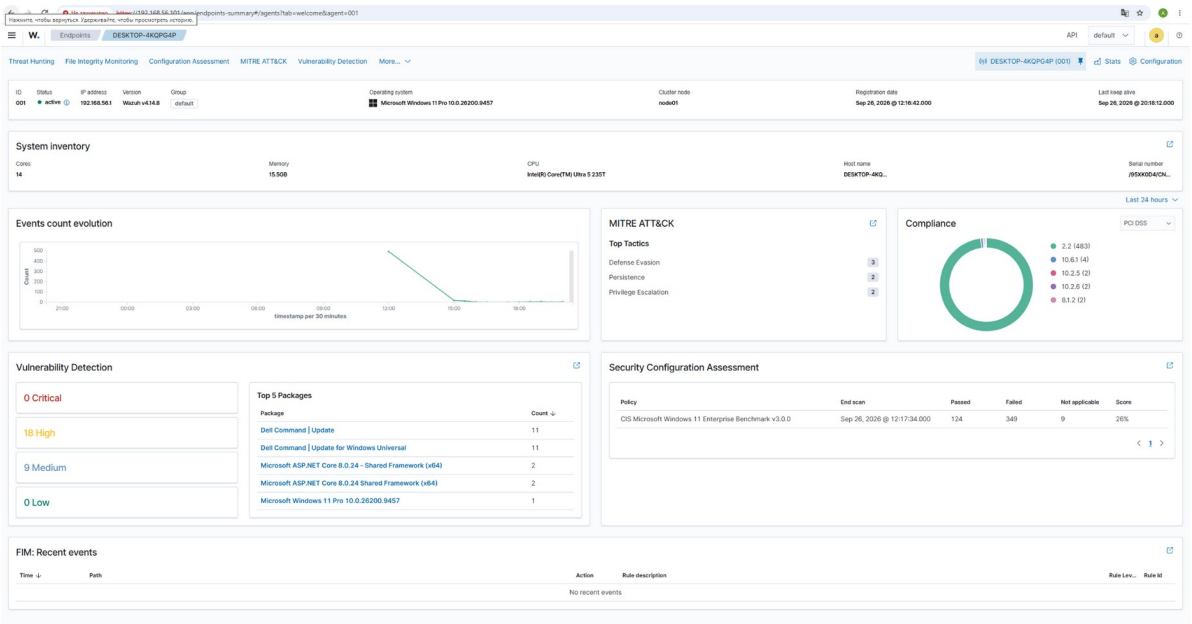


Figure 3.6. Windows agent overview after initial enrollment.

On Windows, the Security event channel supplied authentication events. Event ID 4624 appeared in Wazuh from agent 001; the controlled failed-logon attempt produced Event ID 4625 on the host and a corresponding Wazuh alert. The account, logon type and event time distinguish the controlled attempt from background Windows activity (Wazuh, n.d.-d).

On Ubuntu, Wazuh collected SSH messages through journald. The event detail fields identify the destination account labuser and source address 192.168.56.1 in both failed

and successful SSH events. A separate level 10 alert reflects repeated password failures reported by the SSH/PAM log. These records should not be counted as independent login attempts merely because several Wazuh rules can respond to related log messages (Wazuh, n.d.-d; Wazuh, n.d.-e).

A collected event and an alert are different checks. Wazuh rules generate alerts when their conditions are met; a quiet dashboard after a benign login cannot establish that the event was never collected. To investigate a missing alert, the native endpoint record and the agent's collection settings should be checked first. If analysis of events that do not raise alerts is needed, Wazuh documents a configurable JSON archives option; its `logall_json` setting is disabled by default. Enabling archives changes storage use and will be documented if it becomes necessary during the implementation (Wazuh, n.d.-i; Wazuh, n.d.-j).

Wazuh also provides `wazuh-logtest` for examining which decoder and rule would match a supplied example log. The tool can help explain a rule decision during setup, but it is not a substitute for a live attempt collected from a monitored endpoint. The end-to-end verification for this thesis has three linked observations: a native event on the target machine, evidence of collection or analysis in Wazuh, and a matching alert where one is expected. If a custom rule is later introduced, the rule file, conditions and test method will be documented separately from standard Wazuh behaviour (Wazuh, n.d.-k; Wazuh, n.d.-l).

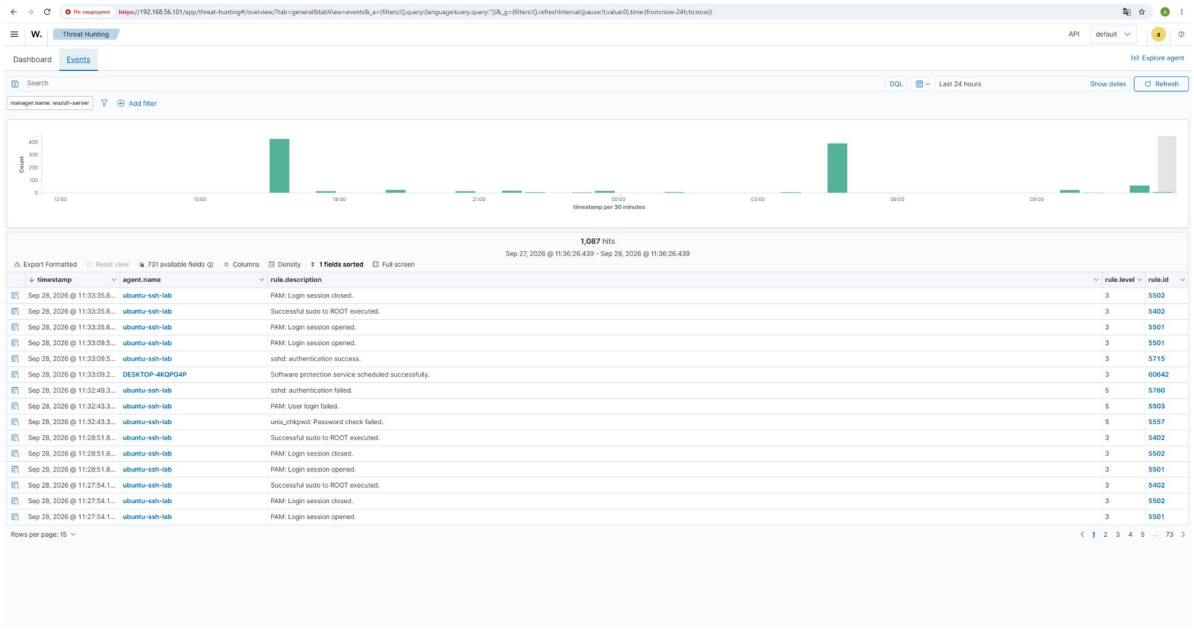


Figure 3.7. Wazuh event list showing Windows and Ubuntu agent records.

3.4 Test Scenarios and Evaluation Criteria

The test design distinguishes normal successful authentication, repeated incorrect passwords and success after earlier failures on each operating system. In the completed laboratory evidence, Windows supplies a controlled failed local logon and separate successful-logon examples; Ubuntu supplies several incorrect SSH passwords followed by a correct SSH login. The six platform–scenario combinations in Table 4 are a comparison framework, not a claim that every combination was run.

Scenario	Planned action on each endpoint	Evidence sought	Expected interpretation
N1 Normal success	One authorised login with correct credentials.	Windows 4624 or Linux SSH acceptance; any relevant Wazuh record.	Valid baseline. An alert, if any, is not automatically an attack.
F1 Repeated failures	Several consecutive incorrect credentials for the same test account from one known source.	Native failures and any Wazuh single-event or repeated-failure alerts.	Determine whether the defined burst is recognised.
S1 Failures then success	A defined failed sequence followed by one authorised success for the same account.	Both outcomes, order and available source details in native logs and Wazuh.	Check whether an analyst can reconstruct the sequence; do not infer compromise.

Table 4. Authentication scenario framework and evidence required for interpretation.

The Windows failed-logon test generated a native 4625 record and Wazuh alert rule 60122. Windows 4624 examples confirm successful-logon visibility but have different logon types and should not be described as a verified failure-to-success sequence for the same Windows account. On Ubuntu, Wazuh rule 5760 records a failed SSH authentication, rule 2502 at level 10 records repeated password failures, and rule 5715 records a subsequent successful SSH authentication. The failed and successful SSH details both name labuser and source 192.168.56.1. Their sequence is an analyst comparison of separate alerts; the screenshots do not demonstrate a single

Wazuh rule that correlates the complete failed-then-successful sequence (Wazuh, n.d.-m).

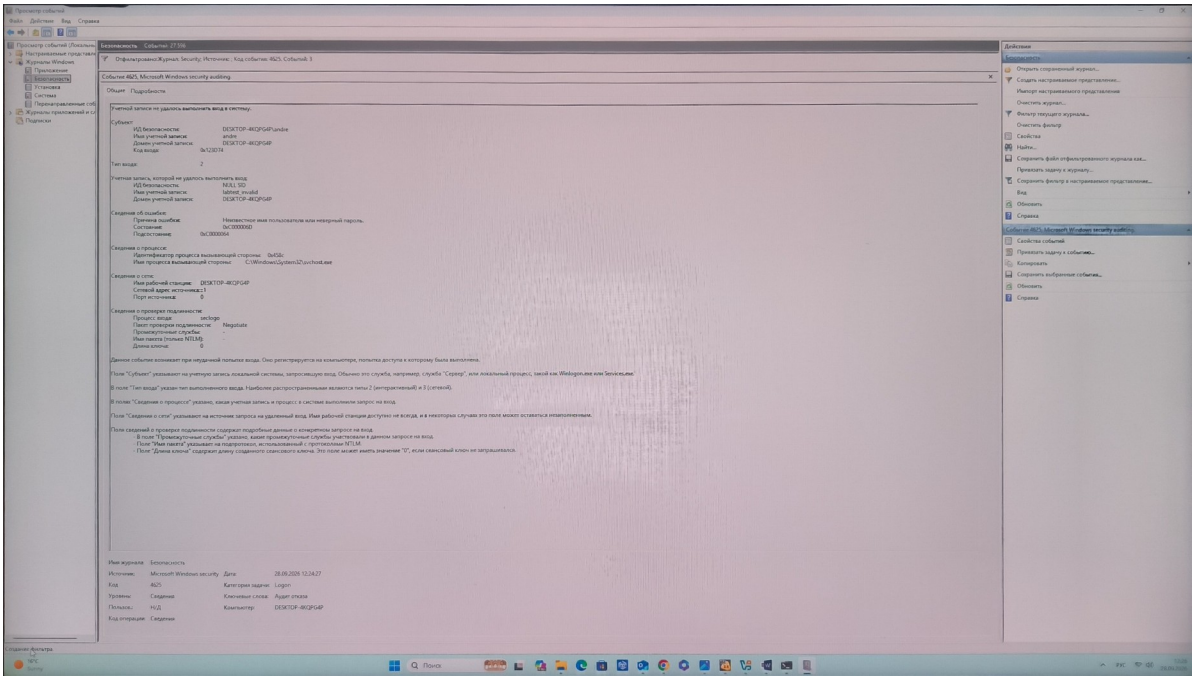


Figure 3.8. Native Windows Security event 4625 for the controlled failed logon.

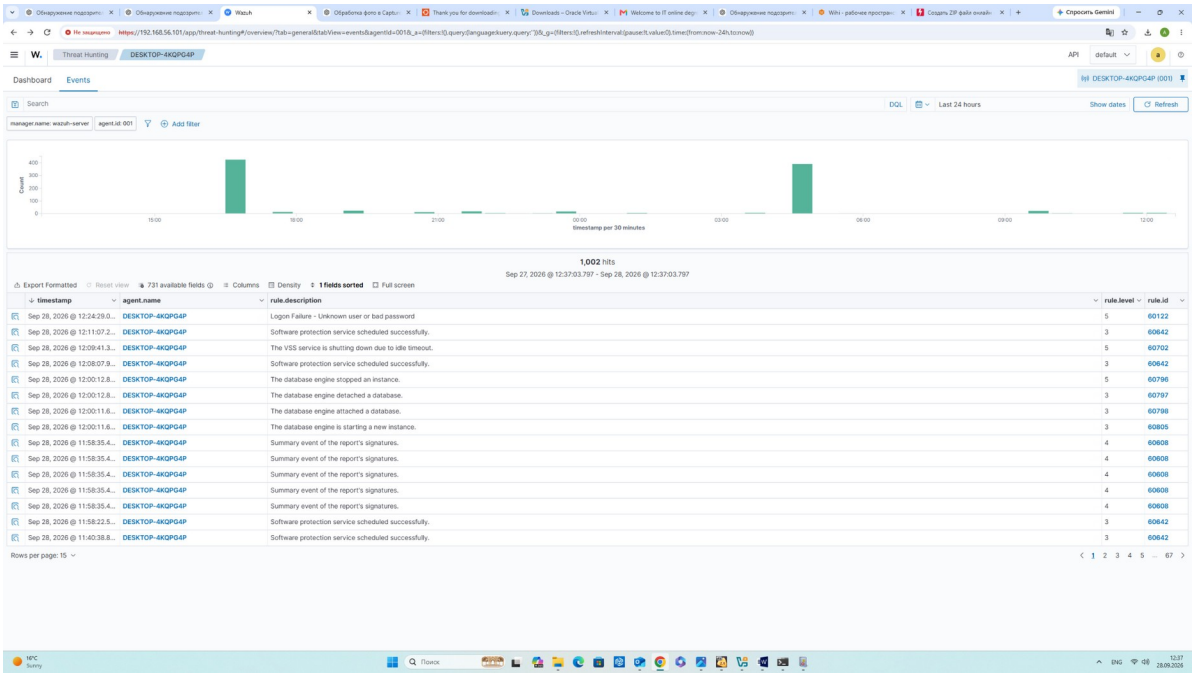


Figure 3.9. Corresponding Windows failed-logon alert in Wazuh, rule 60122.

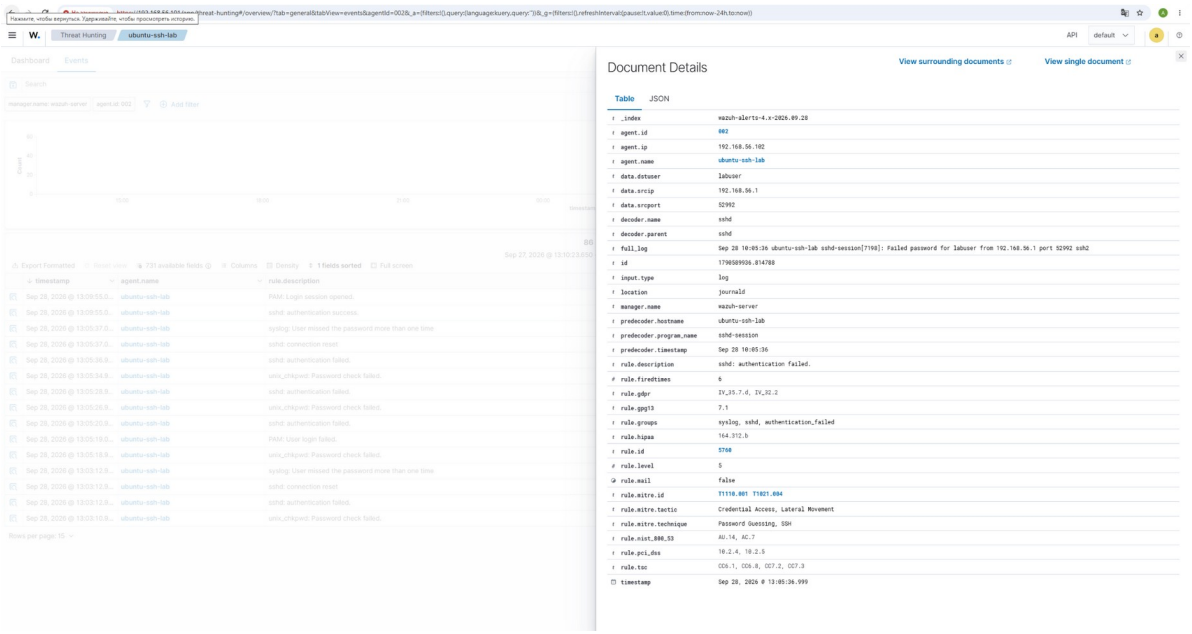


Figure 3.10. Failed SSH authentication for labuser from 192.168.56.1, rule 5760.

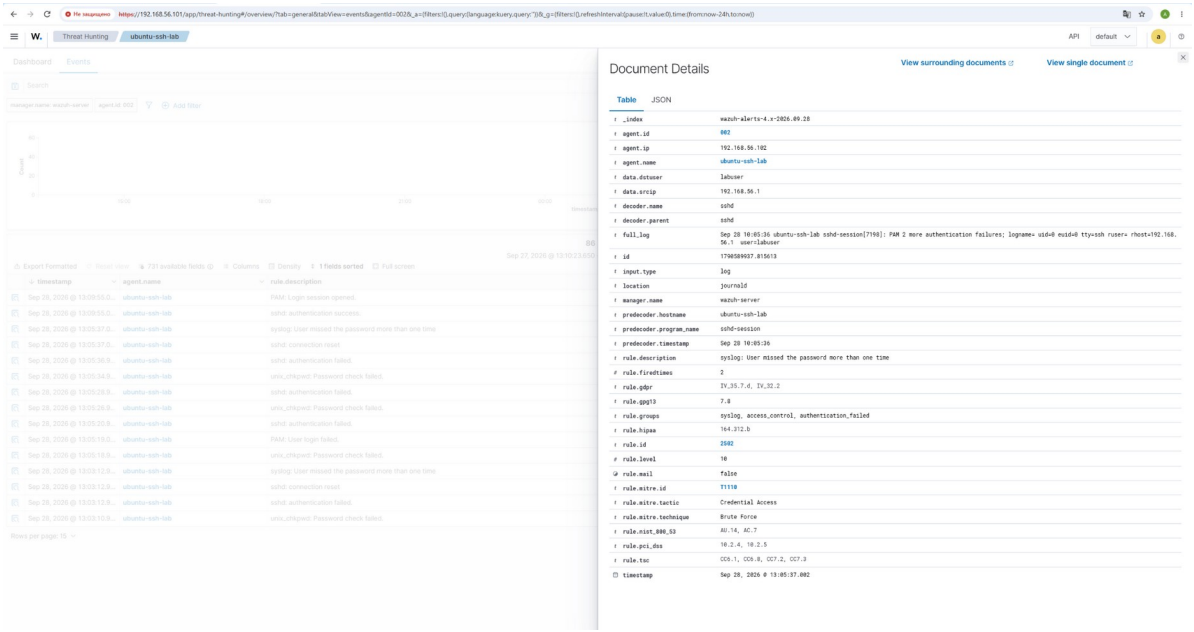


Figure 3.11. Repeated SSH password failures recorded by Wazuh, rule 2502, level 10.

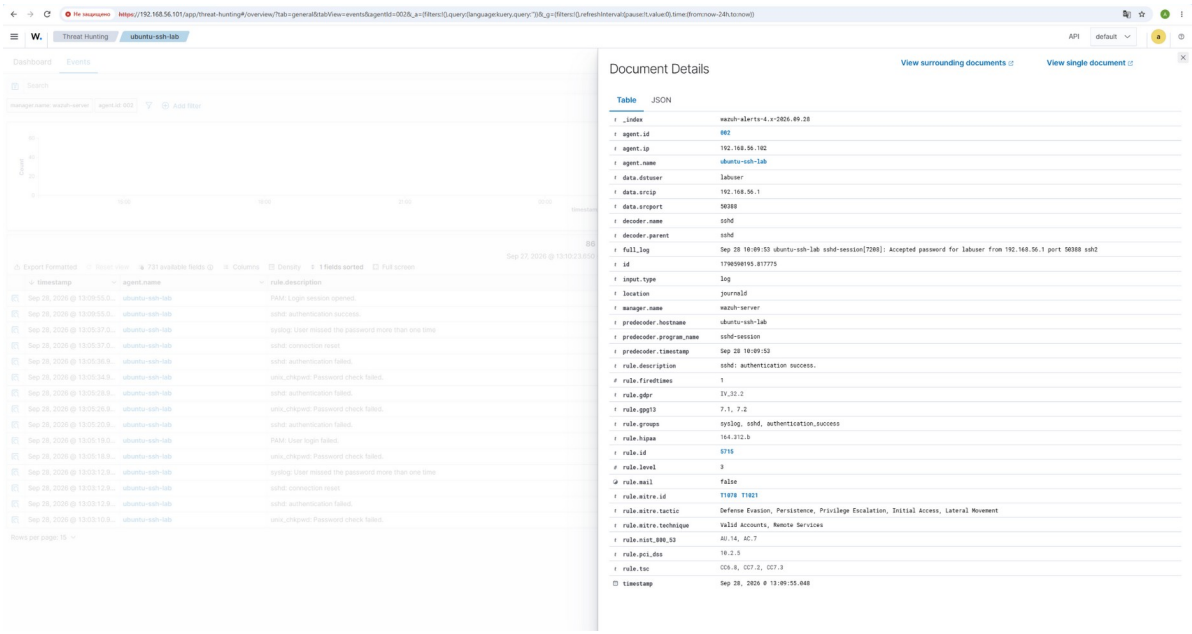


Figure 3.12. Subsequent successful SSH login for labuser from 192.168.56.1, rule 5715.

The test record should include platform, account label, source, method, event times, native records and Wazuh rule IDs. The following figures retain the dashboard context and rule details for the observations. Screenshots included in a public thesis should be reviewed for personal identifiers; passwords and private keys are never included.

Evaluation proceeds at two levels. First, source visibility asks whether the intended authentication actions generated identifiable native records and whether the relevant data was collected for analysis. Second, detection asks whether an alert consistent with the planned pattern appeared under the installed Wazuh configuration. A missing event in Wazuh is a collection or visibility problem until the contrary is shown; a collected event with no alert can be a rule or threshold issue. These distinctions identify where a scenario failed instead of assigning every missing alert to the detection engine.

For each planned pattern, the outcome will be classified as detected, partially detected, not detected or inconclusive. “Detected” requires an identifiable Wazuh alert matching the planned pattern and the correct endpoint. “Partially detected” means constituent events or simpler alerts are available but the planned sequence is not recognised as expected. “Not detected” means adequate native and collected evidence exists yet the expected alert does not appear within the agreed observation period. “Inconclusive” is reserved for missing or ambiguous native evidence, collection

problems, an interrupted test or incompatible timestamps. The observation period and any threshold changes will be stated before the results are interpreted.

A simple detection count can be reported as the number of detected planned scenarios divided by the number of evaluable planned scenarios. Because the experiment is small and deliberately constructed, this fraction describes only the tested scenarios; it is not an estimate of field performance or a population-wide detection rate. Partial and inconclusive outcomes should be shown separately rather than silently added to either the successes or failures. The same caution applies to alert counts: one failed login can produce multiple related alerts, while a correlation rule can summarise several attempts in one alert.

A benign activity can be marked for review if the system labels it with a severity or description suggestive of an attack. In this thesis, a possible false positive means an alert that would incorrectly classify a known benign scenario according to the stated test criteria. The author should compare the rule description and event context before using that term: a low-level event alert accurately stating that a login succeeded is not necessarily a false positive. A possible missed detection means a planned suspicious pattern occurred and is visible in source evidence but the configured detection did not report it according to the prespecified criterion.

The completed laboratory demonstrates selected Windows and Linux authentication events under a small, controlled configuration. It contains neither Active Directory nor representative enterprise traffic, and it does not establish a statistical detection rate or confirm a compromise. Chapter 4 evaluates the observed alerts against the scenario criteria and discusses these limitations.

References used in Chapter 3

Microsoft (2021a). 4624(S): An account was successfully logged on. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4624

Microsoft (2021b). Audit Logon. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/audit-logon

Microsoft (2026). 4625(F): An account failed to log on. https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4625

Ubuntu (n.d.). sshd_config manual page. https://manpages.ubuntu.com/manpages/noble/man5/sshd_config.5.html

Wazuh (n.d.-a). Quickstart. https://documentation.wazuh.com/current/quickstart.html

Wazuh (n.d.-b). Components. https://documentation.wazuh.com/current/getting-started/components/index.html

Wazuh (n.d.-c). How it works. https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/how-it-works.html

Wazuh (n.d.-d). Configuring log collection for different operating systems. https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html

Wazuh (n.d.-e). Journald log collection. https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/journald.html

Wazuh (n.d.-f). Deploying Wazuh agents on Linux endpoints. https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-linux.html

Wazuh (n.d.-g). Deploying Wazuh agents on Windows endpoints. https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html

Wazuh (n.d.-h). Wazuh agent enrollment. https://documentation.wazuh.com/current/user-manual/agent/agent-enrollment/index.html

Wazuh (n.d.-i). Rules syntax. https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html

Wazuh (n.d.-j). Global configuration: logall_json. https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/global.html

Wazuh (n.d.-k). Testing decoders and rules. https://documentation.wazuh.com/current/user-manual/ruleset/testing.html

Wazuh (n.d.-l). Custom rules. https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html

Wazuh (n.d.-m). Detecting a brute-force attack. https://documentation.wazuh.com/current/proof-of-concept-guide/detect-brute-force-attack.html

4 Implementation and Testing

This chapter describes the authentication tests performed on 11 October 2026 in the laboratory introduced in Chapter 3. Windows logon events and Linux SSH records were compared with alerts in Wazuh. The tests confirmed visibility of successful and failed authentication events on both endpoints. Linux also produced an alert for a PAM message reporting repeated authentication failures. A successful login following failures was reconstructed from the records; a dedicated alert correlating that sequence was not demonstrated.

4.1 Verification of Log Collection

The monitored Windows endpoint was DESKTOP-4KQPG4P, agent 001, at 192.168.56.1. It also acted as the SSH client and VirtualBox host. The Ubuntu endpoint was ubuntu-ssh-lab, agent 002, at 192.168.56.102. The Wazuh server was accessed at 192.168.56.101. These addresses distinguish the monitored target from the client that initiated each SSH connection.

Collection was checked by locating an authentication record in the native endpoint log and then finding the corresponding record in Wazuh Threat Hunting. For Windows, the comparison used the target account, event ID, event record ID and source systemTime. For Linux, it used the account, source address, port and full_log. This established event visibility without treating an active agent status alone as proof that the required logs were collected.

The Windows search selected the dedicated test account and the two authentication event IDs:

data.win.eventdata.targetUserName: "wazuh_test" AND
(data.win.system.eventID: "4624" OR data.win.system.eventID: "4625")



Figure 4.1. Wazuh results for wazuh_test showing three failures and one success.

The four returned alerts comprised three failures under rule 60122 and one success under rule 60118. Subsequent inspection of the individual records confirmed that the events belonged to wazuh_test. Windows records were decoded as windows_eventchannel and collected through EventChannel; the SSH records were decoded as sshd and showed journald as their source location.4.2 Normal Login Activity

Windows successful authentication

A local account named wazuh_test was created for the Windows test. Successful authentication was performed by supplying its correct credentials to Start-Process and launching Notepad. This tested local interactive authentication without requiring a full desktop sign-out. The resulting Security event was 4624 with logon type 2.

The source event time was 11:27:07.8411044 UTC, equivalent to 14:27:07 at UTC+3. Wazuh retained the target account, event ID 4624 and event record ID 1097111, as shown in Figure 4.2. The result list in Figure 4.1 associated the successful event with rule 60118 at level 3. This successful event also served as the final step of the Windows failure-then-success test described in Section 4.4; it was not a separate independent baseline trial.



Figure 4.2. Collected Windows event 4624 for the local account wazuh_test.Linux successful authentication

A normal SSH login was initiated from Windows PowerShell with the following command. The correct password was entered at the prompt; passwords were not included in the evidence.

ssh labuser@192.168.56.102

After the connection opened, whoami returned labuser and hostname returned ubuntu-ssh-lab. The native SSH journal recorded Accepted password at 12:59:13 UTC from 192.168.56.1, port 61130. Wazuh preserved the same user, source address, port and original log message. Rule 5715 generated a level 3 alert with the description sshd: authentication success, as shown in Figure 4.3.

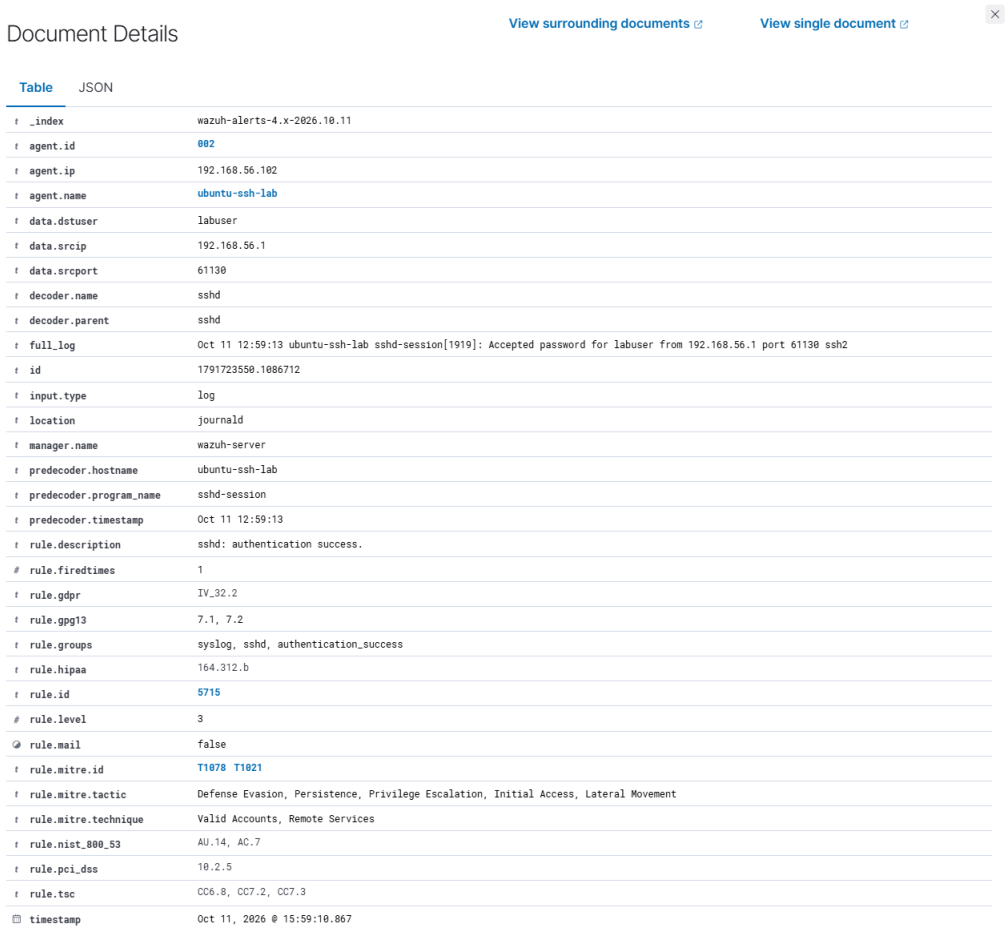


Figure 4.3. Normal SSH authentication from Windows reported by Wazuh rule 5715.

The alert reports an authentication outcome. Its MITRE ATT&CK metadata does not establish malicious activity in this controlled, authorised login.4.3 Repeated Failed Login Attempts

Windows failed authentication

Incorrect credentials were deliberately supplied for wazuh_test when attempting to launch Notepad. PowerShell reported an incorrect user name or password, and Windows recorded Security event 4625. Three failed attempts were visible in the filtered Wazuh results. The account remained the same for the subsequent successful authentication.

The last failure occurred at 11:25:32.2099881 UTC and had event record ID 1097105. The native record showed status 0xC000006D and substatus 0xC000006A. Its logon type was 2, the source address was the loopback address ::1, and the caller process was svchost.exe. These fields describe the local test and should not be interpreted as a remote Windows login from another machine.

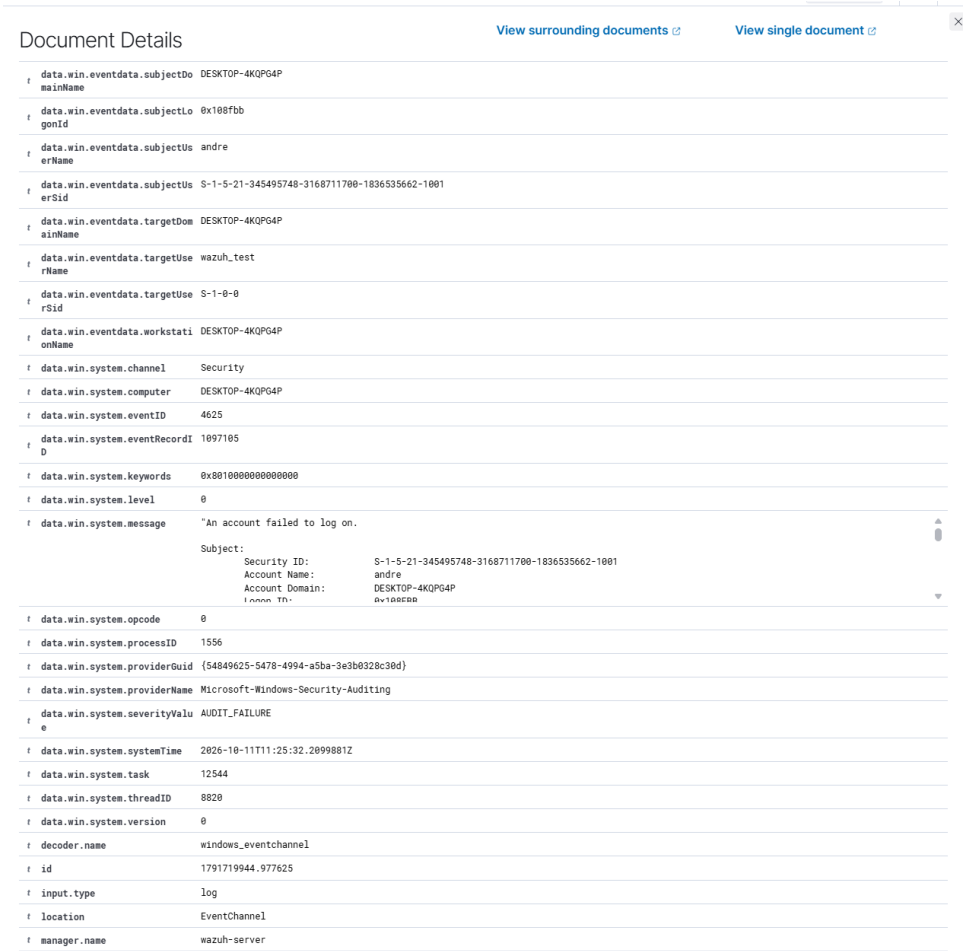
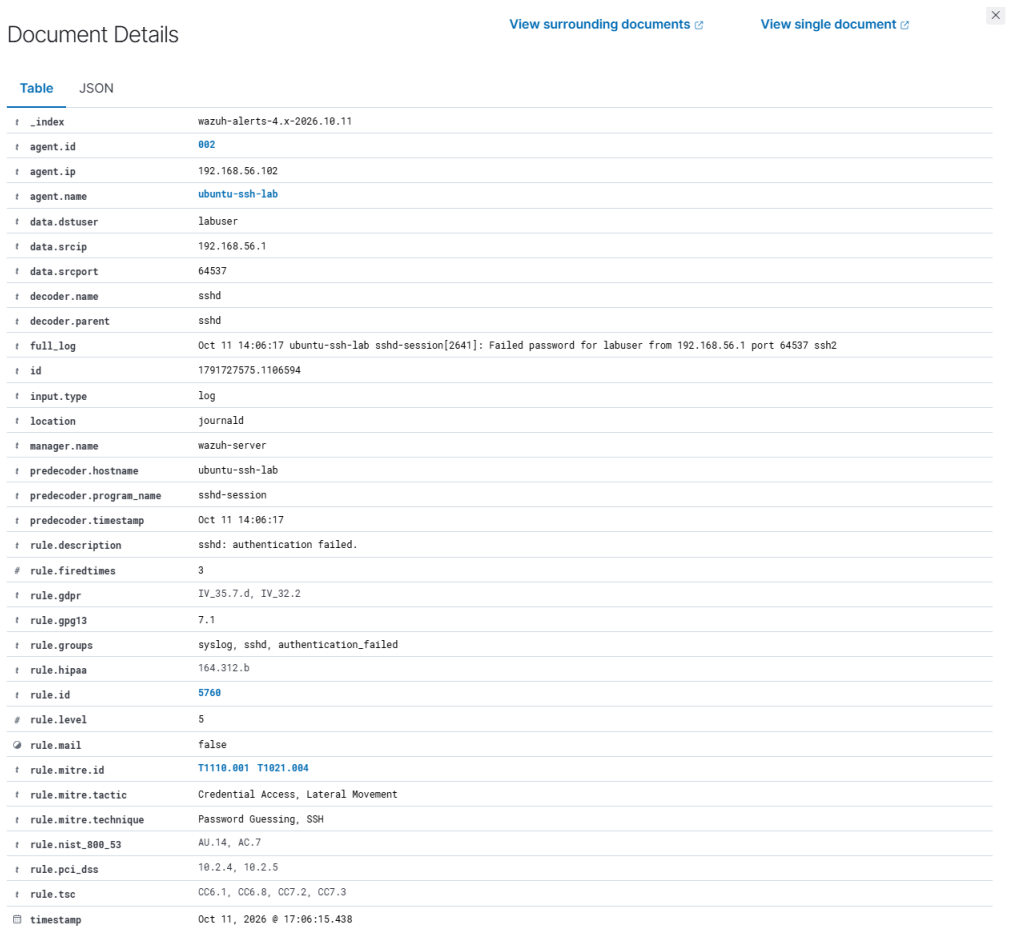


Figure 4.4. Windows event 4625 for wazuh_test with the source event time and record ID.

Wazuh classified the failed attempts under rule 60122, level 5. The observed results establish detection of the individual failures; they do not demonstrate a separate Windows alert for the repeated-failure pattern.Linux failed authentication

For the Linux test, a new SSH connection was initiated at a Windows PowerShell prompt. Three incorrect passwords were entered for labuser. The client reported Permission denied and eventually closed the authentication attempt. The intended source was 192.168.56.1; earlier exploratory connections originating from Ubuntu itself at 192.168.56.102 were excluded from this scenario.

The Ubuntu journal recorded three Failed password messages at 14:06:05, 14:06:12 and 14:06:17 UTC. All three belonged to sshd-session process 2641 and used source port 64537. Wazuh showed three rule 5760 alerts at level 5. Figure 4.5 presents the last failed attempt and preserves the user, address and port needed to match it to the native journal.



Document Details

View surrounding documents | View single document

Table | JSON

_index	wazuh-alerts-4.x-2026.10.11
agent.id	002
agent.ip	192.168.56.102
agent.name	ubuntu-ssh-lab
data.dstuser	labuser
data.srcip	192.168.56.1
data.srcport	64537
decoder.name	sshd
decoder.parent	sshd
full_log	Oct 11 14:06:17 ubuntu-ssh-lab sshd-session[2641]: Failed password for labuser from 192.168.56.1 port 64537 ssh2
id	1791727575.1106594
input.type	log
location	journald
manager.name	wazuh-server
predecoder.hostname	ubuntu-ssh-lab
predecoder.program_name	sshd-session
predecoder.timestamp	Oct 11 14:06:17
rule.description	sshd: authentication failed.
rule.firedtimes	3
rule.gdpr	IV_35.7.d, IV_32.2
rule.gpg13	7.1
rule.groups	syslog, sshd, authentication_failed
rule.hipaa	164.312.b
rule.id	5760
rule.level	5
rule.mail	false
rule.mitre.id	T1110.001 T1021.004
rule.mitre.tactic	Credential Access, Lateral Movement
rule.mitre.technique	Password Guessing, SSH
rule.nist_800_53	AU.14, AC.7
rule.pci_dss	10.2.4, 10.2.5
rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
timestamp	Oct 11, 2026 @ 17:06:15.438

Figure 4.5. Rule 5760 for the third failed SSH password from 192.168.56.1.Alert for repeated password failures

After the three failed passwords, Ubuntu generated a PAM summary at 14:06:19 UTC stating PAM 2 more authentication failures. The message identified labuser and rhost=192.168.56.1. Wazuh produced rule 2502 at level 10 with the description User missed the password more than one time. Figure 4.6 shows the original summary and the alert fields.

This result confirms an alert for the repeated-failure information contained in a PAM message. It does not by itself demonstrate that Wazuh correlated three separate rule 5760 alerts. The controlled test also does not establish an actual brute-force attack: the attempts were authorised, while the rule metadata associated the event with MITRE ATT&CK T1110.

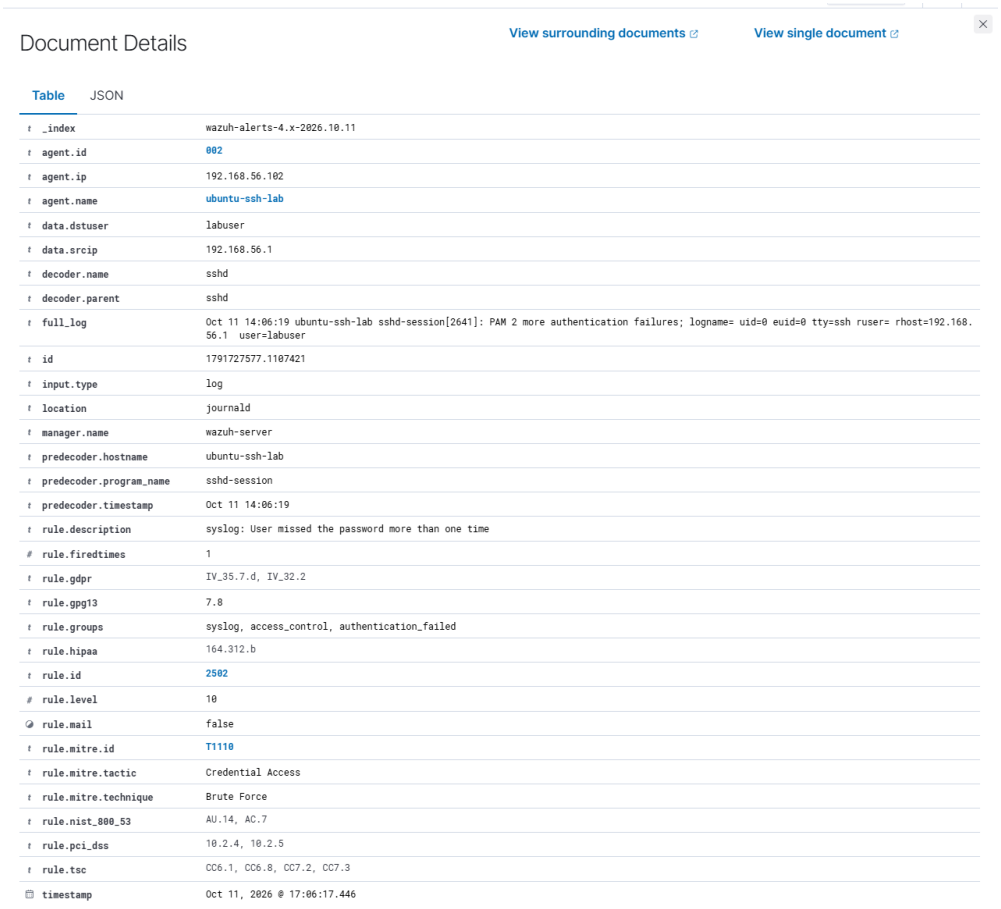


Figure 4.6. Rule 2502 generated from the PAM summary of repeated authentication failures.

4.4 Successful Login Following Failed Attempts

The next scenario examined whether failed authentication records and a later successful login could be identified for the same account. The comparison used source event times rather than relying solely on the order of alert timestamps in the dashboard.

For Windows, the three failures for wazuh_test were followed by successful authentication at 11:27:07 UTC. The last failure at 11:25:32 UTC preceded the success by approximately 96 seconds. The matching target account and event records supported this reconstruction. No dedicated Windows failure-then-success correlation alert was demonstrated.

For Linux, the same Windows client subsequently authenticated successfully as labuser. The SSH service recorded Accepted password at 14:13:59 UTC from 192.168.56.1, port 57473. The success occurred 7 minutes and 42 seconds after the last Failed password message. It opened a new SSH connection, so its source port differed from the failed connection. Wazuh classified it under rule 5715, level 3.

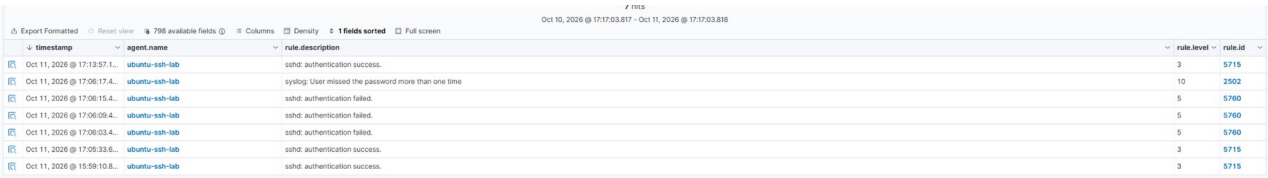


Figure 4.7. Linux alerts showing failures, the PAM summary and a later successful login.

The query used to compare the relevant Linux alerts was:

agent.id: "002" AND data.srcip: "192.168.56.1" AND
(rule.id: "5760" OR rule.id: "2502" OR rule.id: "5715")

The result list also contains successful logins before the failure sequence. Those earlier records were not counted as the subsequent success. The specific later event was verified by opening its details and matching its full_log time, user and source address, as shown in Figure 4.8.

Verification of the subsequent successful SSH login

Figure 4.8 identifies agent 002 as the target, labuser as the authenticated account and 192.168.56.1 as the client. The original log records the successful authentication at 14:13:59 UTC. This is the successful event used in the Linux failure-then-success scenario, rather than one of the earlier normal logins.

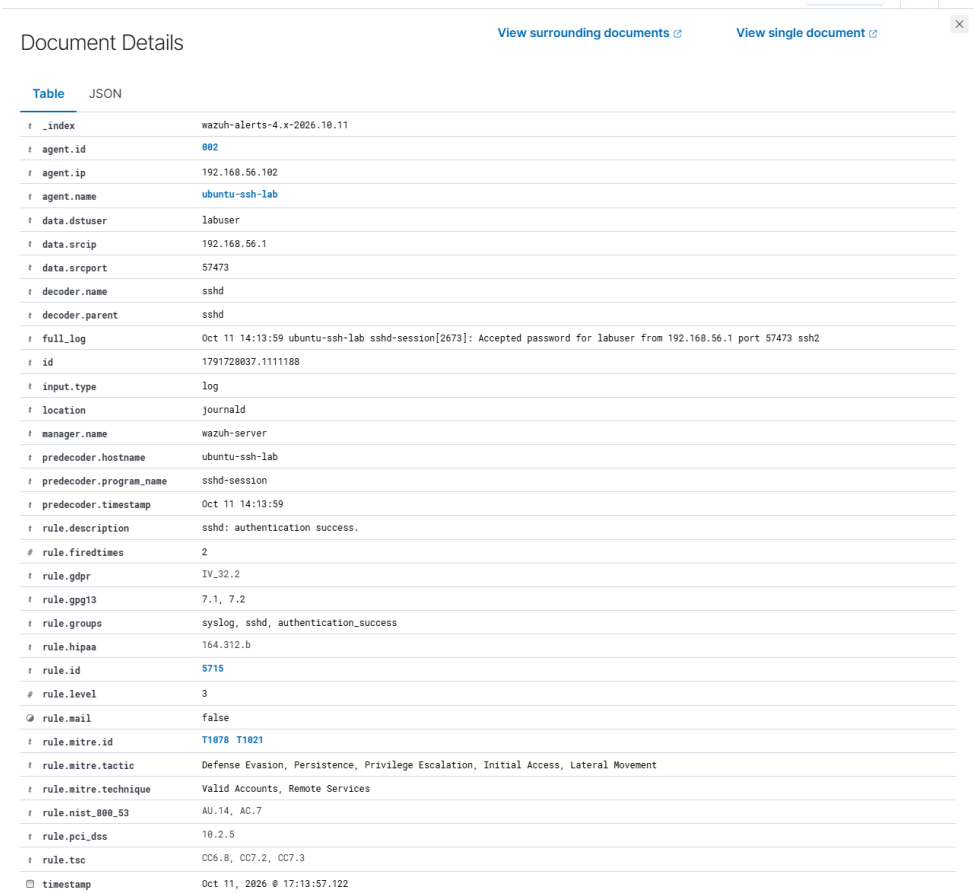


Figure 4.8. Subsequent successful SSH authentication reported by rule 5715.

The observations demonstrate that the component events were available for investigation. Linking them into a suspicious sequence required manual comparison. A level 3 successful-login alert alone does not establish that Wazuh detected the complete failure-then-success pattern.

Comparison with the original Ubuntu journal

The source journal was retrieved over the active SSH session for the interval 14:05 to 14:15 UTC on 11 October 2026. This interval includes an earlier successful login, the controlled failures, their PAM summary and the later success. Figure 4.9 shows the native records used to verify the Wazuh observations.

```
sudo journalctl -u ssh.service \
  --since "2026-10-11 14:05:00" \
  --until "2026-10-11 14:15:00" --no-pager -o short-iso
```



Figure 4.9. Native Ubuntu journal confirming three failed passwords and the later success.

Table 4.1. Source chronology for the controlled Linux test on 11 October 2026.

UTC time	Native event	Wazuh rule	Level
14:06:05	Failed password for labuser	5760	5
14:06:12	Failed password for labuser	5760	5
14:06:17	Failed password for labuser	5760	5
14:06:19	PAM summary of additional failures	2502	10
14:13:59	Accepted password for labuser	5715	3

Every event in Table 4.1 originated from 192.168.56.1. Matching the records by user, address, port and message avoided confusing this test with Ubuntu-to-itself SSH attempts. The journal confirms source event ordering; the Wazuh details confirm collection and the associated alert classifications.

The Ubuntu journal used UTC, while the dashboard displayed times at UTC+3. After accounting for this time-zone difference, some Linux alert timestamps still preceded the source timestamps by about two seconds. The Windows dashboard also displayed the test events together at approximately 14:59 despite their earlier source

systemTime values. Clock alignment and collection timing were not established sufficiently to calculate reliable ingestion latency.4.5 Testing Detection Rules

Rule testing was performed by generating controlled authentication outcomes and inspecting the resulting Wazuh alerts. The evidence covers the installed rules exercised by these scenarios. No custom correlation rule was implemented or validated in this test series, and no separate replay test using wazuh-logtest was performed.

Table 4.2. Rules observed during the Windows and Linux authentication tests.

Platform	Rule	Level	Observed outcome
Windows	60118	3	Successful authentication for wazuh_test
Windows	60122	5	Individual failed logins for wazuh_test
Linux	5715	3	Accepted password for labuser
Linux	5760	5	Individual Failed password messages
Linux	2502	10	PAM summary reporting additional failures

The Windows result list supplied the rule IDs and levels, while individual event details supplied the target account, event ID, source time and record ID. The Linux detail views supplied both the original message and the relevant rule fields. The comparison therefore distinguished evidence that an action generated a source event, evidence that Wazuh collected it, and evidence that a particular rule produced an alert.

The three controlled Windows failures and the corresponding success were visible in the selected Wazuh results. The three Linux Failed password records were also represented by three rule 5760 alerts, followed by rule 2502 for the PAM summary and rule 5715 for the later accepted password. Within these selected scenarios, no missing component authentication record was identified. This finding is limited to the checked records and does not establish a general detection rate.

Normal successful authentication generated low-level alerts in both systems. An alert accurately reporting a successful login was not treated as a false positive merely because it appeared in the security dashboard. Likewise, attack-related metadata on an authorised test event was interpreted in the context of the known test actions.

The failure-then-success scenario remained a separate evaluation question. Its component events were visible, but the screenshots did not demonstrate an automatic alert joining those events into one suspicious pattern. The Linux level 10 alert referred to repeated failures, not to the later successful login. The same distinction applies to the individual Windows failure and success alerts.

The tests provide a reproducible example of event visibility and rule classification on two endpoints. They do not cover threshold boundaries, repeated independent trials, production traffic, account lockout behaviour or all login types. Chapter 5 discusses the Windows and Linux outcomes, the limits of false-positive and missed-detection assessment, and the implications of the observed timestamp differences.

5 Results and Discussion

The tests described in Chapter 4 showed that Wazuh collected and classified successful and failed authentication events from Windows and Ubuntu. The collected fields supported comparison with the original endpoint logs and reconstruction of a successful login after failed attempts. Linux also produced a level 10 alert for a PAM message reporting repeated failures. The evidence supports these specific outcomes; it does not establish automatic detection of the complete sequence or a general detection rate.

5.1 Windows Detection Results

The Windows results demonstrate visibility of local authentication for the dedicated account wazuh_test. The filtered Wazuh view contained three failed attempts and one successful authentication. Failures were associated with rule 60122 at level 5, while success was associated with rule 60118 at level 3. Figures 4.1, 4.2 and 4.4 provide the result list and individual event details. Together they connect the generated test actions to identifiable Windows Security events and Wazuh alerts.

The event fields helped distinguish the test from unrelated background activity. The last failed attempt was event 4625, record 1097105, at 11:25:32.2099881 UTC. It identified wazuh_test as the target and retained status 0xC000006D and substatus 0xC000006A. The successful event was 4624, record 1097111, at 11:27:07.8411044 UTC. Matching the target account and record identifiers gave stronger evidence of collection than an agent status or an overall event count alone.

Both records concerned local interactive authentication with logon type 2. The loopback address ::1 and the caller process svchost.exe were consistent with the local credential test. The result therefore applies to the tested launch of Notepad with supplied credentials. It cannot be extended to Remote Desktop, network logons or domain authentication, which were not exercised in this series.

The successful event occurred approximately 96 seconds after the last failure. This established the order of the selected records for the same account. However, the observed alerts classified the individual outcomes, and no separate alert linking the failures to the later success was demonstrated. The same success was used in the normal-login discussion and the sequence analysis, so those descriptions represent one observation rather than two independent trials.

5.2 Linux Detection Results

Linux provided evidence for successful SSH authentication, individual password failures and a summary of repeated failures. The normal login at 12:59:13 UTC was

initiated from Windows at 192.168.56.1 and accepted for labuser on ubuntu-ssh-lab. Wazuh retained the source address, port 61130 and original SSH message under rule 5715 at level 3. This supported verification of collection from journald for the tested SSH activity, as shown in Figure 4.3.

Table 5.1. Observed Linux authentication alerts

Source record	Rule	Level
Accepted password	5715	3
Failed password	5760	5
PAM summary of additional failures	2502	10

The controlled failure sequence contained three Failed password messages at 14:06:05, 14:06:12 and 14:06:17 UTC. The shared source address, port 64537 and sshd-session process 2641 linked these records to one SSH connection. Wazuh represented the failures with three rule 5760 alerts. The PAM summary at 14:06:19 UTC then produced rule 2502 at level 10, as documented in Figures 4.5 and 4.6.

Rule 2502 responded to a source message that already reported additional authentication failures. Its appearance demonstrates detection of that summary message. It does not establish that Wazuh independently counted and correlated the three preceding rule 5760 alerts. This distinction matters when assessing what the installed rules actually detected. The level 10 result also describes the configured alert severity, rather than a measured probability that an attack occurred.

A later Accepted password message at 14:13:59 UTC identified the same account and Windows source, using a new connection on port 57473. It followed the last failure by 7 minutes and 42 seconds and produced rule 5715 at level 3. The native journal in Figure 4.9 and the details in Figure 4.8 supported manual reconstruction of the sequence. No additional alert identifying the later success as part of that sequence was demonstrated.

5.3 False Positives and Missed Events

The assessment distinguishes an accurate authentication alert from a conclusion that malicious activity occurred. Successful logins produced level 3 alerts on both systems, including the authorised SSH baseline. These alerts correctly described an authentication outcome. Their presence in Wazuh was therefore not sufficient reason to classify them as false positives. A false-positive assessment would require a

defined detection objective and a decision about whether the alert incorrectly satisfied that objective.

The same caution applies to the deliberate password failures. They were authorised laboratory actions that generated genuine failure records. Wazuh reporting these failures was consistent with the source logs. The MITRE ATT&CK metadata attached to SSH rules supplied a security classification, but it did not establish attacker intent or account compromise. In this test, the known actions and the original records provided the context needed to interpret the alerts.

No missing component authentication record was identified in the selected Windows and Linux sequences. The checked Windows results contained the three failures and subsequent success. The checked Linux records contained the three password failures, their PAM summary and the later success. This supports event visibility for those observations. It does not justify a claim of 100 percent detection, because the experiment did not establish a complete population of authentication activity or test all relevant scenarios.

The sequence of failures followed by success exposes a separate evaluation issue. Its component records were available, but a dedicated correlation alert was not demonstrated. The Linux level 10 alert concerned repeated failures and preceded the later successful login. It cannot serve as evidence that Wazuh recognised that later login as suspicious. The Windows results likewise showed individual event alerts. These observations identify a gap in the demonstrated sequence-level behaviour, while leaving the availability of other rules or configurations untested.

The evidence also has limits as a basis for calculating false-positive and false-negative rates. There was no labelled production dataset, extended baseline period or repeated set of independent trials. An event-level collection check and an incident-level detection assessment use different units, and their results should not be combined into one percentage. Future evaluation could define an expected alert for each scenario, record all generated actions and repeat the trials under controlled timing conditions. That would allow missed events and unexpected alerts to be assessed against explicit criteria.

5.4 Limitations of the Test Environment

The laboratory consisted of one monitored Windows endpoint, one Ubuntu endpoint and a Wazuh server. Windows also hosted VirtualBox and initiated the SSH tests. This arrangement was sufficient to compare endpoint records with centralised alerts, but it represented a small environment with known users and controlled actions. It did

not reproduce concurrent users, enterprise authentication infrastructure or the background activity of a production network. The findings therefore concern the tested configuration and records.

The authentication coverage was limited. Windows tests used a local account and interactive authentication through Start-Process. Linux tests used SSH password authentication for labuser. The experiment did not evaluate account lockout behaviour, alternative Windows logon types, SSH keys or authentication across multiple targets. Three failed passwords in the controlled SSH connection were enough to observe the PAM summary, but they did not measure the response to sustained attempts or establish a detection threshold.

Timing was another constraint. The Ubuntu source journal used UTC, while the Wazuh dashboard displayed UTC+3. Some Linux alert timestamps still preceded the corresponding source timestamps by about two seconds after this conversion. Windows test records appeared together in the dashboard at approximately 14:59 despite earlier source systemTime values. The available evidence did not establish the cause of these differences. Consequently, ingestion latency and real-time detection performance could not be calculated reliably. Source event times were used for the sequence analysis.

The test series examined the installed rules that appeared in the collected results. No custom correlation rule was implemented, and no separate replay validation with wazuh-logtest was performed. The absence of a demonstrated correlation alert should therefore be treated as a limitation of the tested behaviour and evidence. It is not proof that every Wazuh configuration would behave identically. Testing a defined rule for failures followed by success would require its own configuration, expected outcome and validation records.

The evidence was reviewed through selected endpoint logs and Wazuh views. This made individual matches possible, but it did not provide continuous monitoring statistics, resource measurements or an independently verified record of every event generated during the session. Stronger evaluation would retain structured exports alongside the screenshots, verify clock synchronisation and repeat both normal and suspicious scenarios. Those steps would support reliable counts, timing measurements and comparison across trials.

6 Conclusions

This thesis examined whether authentication activity from Windows and Linux could be collected and investigated with Wazuh in a controlled laboratory. The tests established visibility of successful and failed authentication on both endpoints. Comparing Wazuh alerts with Windows Security records and the Ubuntu SSH journal made it possible to identify the tested accounts, verify individual outcomes and reconstruct a successful login following failed attempts.

Windows generated event 4625 for the deliberate password failures and event 4624 for the subsequent successful authentication of wazuh_test. Wazuh classified these outcomes under rules 60122 and 60118. The logon type and loopback address identified the scenario as local interactive authentication. These results support the collection and investigation of the tested activity, while leaving remote Windows authentication outside the evaluated scope.

On Ubuntu, Wazuh reported individual failed SSH passwords under rule 5760 and successful password authentication under rule 5715. A PAM summary of additional failures produced rule 2502 at level 10. The preserved account, source address, port and original message supported comparison with journald. The later successful SSH login could be linked manually to the earlier failures through the same account, target and source, with source timestamps establishing their order.

The experiment also identified a limit in the demonstrated detection behaviour. Individual failures, the Linux summary and successful logins were visible, but no dedicated alert correlating failures with the later success was demonstrated. The PAM summary alert cannot be treated as evidence of that complete correlation. Likewise, a successful-login alert or an ATT&CK label does not by itself establish unauthorised access. The authorised test actions supplied the context needed to interpret the records.

The main practical contribution is a documented method for checking authentication evidence from its native source through to Wazuh. It separates event generation, collection and alert interpretation, helping an investigator identify what a result actually demonstrates. The small number of endpoints and trials, limited authentication methods and unresolved timestamp differences prevent general claims about detection rates, false-positive rates or ingestion latency.

Further work should synchronise and verify clocks, retain structured exports and repeat the scenarios against explicit expected outcomes. A dedicated rule for repeated failures followed by success could then be implemented and tested with both benign and suspicious sequences. The present laboratory establishes a basis for authentication

monitoring and manual investigation, while showing which additional validation is needed before claiming automatic recognition of the full pattern.

References

Microsoft (2021a). 4624(S): An account was successfully logged on.

Available at: https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4624 (Accessed 16 October 2026).

Microsoft (2021b). Audit Logon.

Available at: https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/audit-logon (Accessed 16 October 2026).

Microsoft (2026). 4625(F): An account failed to log on.

Available at: https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-10/security/threat-protection/auditing/event-4625 (Accessed 16 October 2026).

Microsoft (n.d.). Administrative tools and logon types reference.

Available at: https://learn.microsoft.com/en-us/windows-server/identity/securing-privileged-access/reference-tools-logon-types (Accessed 16 October 2026).

MITRE ATT&CK (n.d.). Brute Force, Technique T1110.

Available at: https://attack.mitre.org/techniques/T1110/ (Accessed 16 October 2026).

systemd (n.d.). journalctl. Version 255 manual.

Available at: https://www.freedesktop.org/software/systemd/man/255/journalctl.html (Accessed 16 October 2026).

References continued

Ubuntu (n.d.). sshd_config: OpenSSH daemon configuration file. Ubuntu 24.04 LTS manual.

Available at: https://manpages.ubuntu.com/manpages/noble/man5/sshd_config.5.html (Accessed 16 October 2026).

Wazuh (n.d.-a). Configuring log collection for different operating systems.

Available at: https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html (Accessed 16 October 2026).

Wazuh (n.d.-b). Journald log collection.

Available at: https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/journald.html (Accessed 16 October 2026).

Wazuh (n.d.-c). Log data analysis.

Available at: https://documentation.wazuh.com/current/getting-started/use-cases/log-analysis.html (Accessed 16 October 2026).

Wazuh (n.d.-d). Testing decoders and rules.

Available at: https://documentation.wazuh.com/current/user-manual/ruleset/testing.html (Accessed 16 October 2026).

References continued

Wazuh (n.d.-e). Components.

Available at: https://documentation.wazuh.com/current/getting-started/components/index.html (Accessed 16 October 2026).

Wazuh (n.d.-f). How it works: Log data collection.

Available at: https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/how-it-works.html (Accessed 16 October 2026).

Wazuh (n.d.-g). Rules Syntax.

Available at: https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html (Accessed 16 October 2026).

Wazuh (n.d.-h). Detecting a brute-force attack.

Available at: https://documentation.wazuh.com/current/proof-of-concept-guide/detect-brute-force-attack.html (Accessed 16 October 2026).

Wazuh (n.d.-i). Custom rules.

Available at: https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html (Accessed 16 October 2026).

References continued

Wazuh (n.d.-j). Quickstart.

Available at: https://documentation.wazuh.com/current/quickstart.html (Accessed 16 October 2026).

Wazuh (n.d.-k). Deploying Wazuh agents on Linux endpoints.

Available at: https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-linux.html (Accessed 16 October 2026).

Wazuh (n.d.-l). Deploying Wazuh agents on Windows endpoints.

Available at: https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html (Accessed 16 October 2026).

Wazuh (n.d.-m). Wazuh agent enrollment.

Available at: https://documentation.wazuh.com/current/user-manual/agent/agent-enrollment/index.html (Accessed 16 October 2026).

Wazuh (n.d.-n). global: Local configuration (ossec.conf).

Available at: https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/global.html (Accessed 16 October 2026).

Appendices

Appendix 1. Laboratory Configuration

This appendix records the environment used for the final tests on 11 October 2026. Earlier September examples in Chapter 3 document setup and initial visibility checks; the Windows account and controlled sequences below refer to the later tests in Chapter 4.

Component	Identity and address	Function
Windows host	DESKTOP-4KQPG4P 192.168.56.1 Agent 001	Windows 11 Pro endpoint, VirtualBox host and SSH client

Wazuh VM	192.168.56.101 Manager: wazuh-server	Server, indexer and dashboard on one VM
Ubuntu VM	ubuntu-ssh-lab 192.168.56.102 Agent 002	Ubuntu 26.04.1 LTS endpoint and SSH server

The host-only network carried traffic between these components. The Ubuntu console also reported 10.0.2.15 on its NAT interface; the controlled SSH tests used 192.168.56.102. Wazuh was accessed at https://192.168.56.101/. Agent inventory screenshots reported version 4.14.8. Exact central-component builds, VirtualBox version and VM resource allocations were not retained as verified configuration measurements.

Authentication and collection

Windows used wazuh_test with Start-Process for interactive authentication. Reference records were Security events 4624 and 4625, logon type 2. Wazuh details identified EventChannel collection and the windows_eventchannel decoder. Both outcomes were observed; a complete audit-policy export was not retained.

Ubuntu used labuser with SSH password authentication. Wazuh details identified journald collection and the sshd decoder. Native records were checked with journalctl for ssh.service. Complete ossec.conf and effective sshd configuration exports were not archived.

Source event times are reported in UTC. Dashboard displays used UTC+3. Passwords and private keys are excluded. No custom correlation rule or wazuh-logtest replay formed part of the final test series.

Appendix 2. Test Scenarios and Results

Windows tests

The final Windows test used wazuh_test on agent 001. Three attempts supplied incorrect credentials to launch Notepad, followed by an attempt using correct credentials. These actions produced native Security records and corresponding Wazuh alerts. The successful event also served as the Windows normal-login example; it was not a separate independent baseline trial.

Test	Expected evidence	Observed result
W1	4624 for wazuh_test; collected	4624, type 2; rule 60118, level 3

Local success	success record	
W2 Three failures	4625 records for the test account	Three failures in Wazuh; rule 60122, level 5
W3 Success after failures	Same target account; ordered failure and success records	Sequence reconstructed manually; dedicated correlation alert not demonstrated

Verified native records

The last failure occurred at 11:25:32.2099881 UTC, event 4625, record 1097105. The target was wazuh_test, status was 0xC000006D and substatus was 0xC000006A. The record showed logon type 2, address ::1 and caller process C:\Windows\System32\svchost.exe.

Success occurred at 11:27:07.8411044 UTC, event 4624, record 1097111, for wazuh_test with logon type 2. The interval after the last failure was approximately 96 seconds. Figures 4.1, 4.2 and 4.4 provide the collected result list and details. A service logon by SYSTEM or a cached logon for another account was not substituted for the test success.

The observed outcome was successful visibility and classification of the individual events. The Windows evidence did not demonstrate an alert recognising repeated failures as one pattern or linking them automatically to the later success. These findings apply to the local authentication method tested.

Appendix 2 continued

Linux SSH tests

Windows at 192.168.56.1 initiated SSH password authentication to labuser on 192.168.56.102. A baseline success was verified at 12:59:13 UTC, source port 61130, under Wazuh rule 5715 at level 3. Earlier SSH attempts originating from Ubuntu itself at 192.168.56.102 were excluded from the final controlled sequence.

UTC time	Native observation	Wazuh result
14:06:05	First Failed password	5760, level 5
14:06:12	Second Failed password	5760, level 5

14:06:17	Third Failed password	5760, level 5
14:06:19	PAM 2 more authentication failures	2502, level 10
14:13:59	Accepted password for labuser	5715, level 3

The three failed passwords shared source port 64537 and sshd-session process 2641. The later success used port 57473 and process 2673, reflecting a new connection. All listed events concerned labuser and source 192.168.56.1. The success followed the last failed password by 7 minutes and 42 seconds.

The failure test showed individual failure alerts and an alert for the PAM summary. The summary reports additional failures within the source message and must not be counted as an extra password attempt. Its detection does not prove that Wazuh independently correlated the three rule 5760 alerts.

The later success and preceding failures could be compared manually. A dedicated alert correlating the complete sequence was not demonstrated. Figures 4.5–4.9 and Table 4.1 retain the supporting evidence. No general detection percentage, false-positive rate or ingestion-latency measurement was derived from this single controlled sequence.

Appendix 3. Wazuh Rules and Queries

Rules observed in the final tests

The table records the rule IDs and levels displayed in the final Wazuh results. It describes observed matches rather than reproducing unexported XML. Rule behaviour in other versions or configurations requires separate verification. Rules syntax and custom-rule authoring are documented by Wazuh (n.d.-g; n.d.-i).

Rule	Level	Observed match
60118	3	Windows successful local authentication for wazuh_test
60122	5	Windows failed authentication for wazuh_test
5715	3	SSH password authentication success for labuser
5760	5	Individual failed SSH password for labuser
2502	10	PAM message reporting additional authentication failures

The Windows rules were identified in the filtered result list and linked to the native records through account, event ID and record ID. Linux details retained the original SSH/PAM message, account and source information. Rule 2502 had level 10, while the later accepted password retained rule 5715 at level 3. No additional rule joining those outcomes was demonstrated.

No custom rule was implemented in the evaluated series. No local_rules.xml entry, correlation threshold or configured time window is reported for this series. A future rule would need to define the target endpoint, account and source matching conditions, then be checked against both a failed sequence and benign logins. Wazuh documents replay testing with wazuh-logtest, but that replay was not performed here (Wazuh, n.d.-d).

Appendix 3 continued

Dashboard search queries

The following queries reproduce the filters used to compare the final authentication records in Threat Hunting. They are dashboard search expressions, not Wazuh API WQL commands. Agent IDs identify the monitored target. The source-address condition in the Linux search identifies the client that initiated the attempt.

Windows test account and outcomes

agent.id: "001" AND
data.win.eventdata.targetUserName: "wazuh_test" AND
(data.win.system.eventID: "4624" OR
data.win.system.eventID: "4625")

The account and event-ID expression was used with the Windows agent selected. Adding the explicit agent clause expresses the same target restriction. The result list showed three failed authentications and one successful authentication. Individual details supplied source systemTime and event record ID; the list timestamp alone was not used to order the Windows actions.

Linux client and relevant rules

agent.id: "002" AND data.srcip: "192.168.56.1" AND
(rule.id: "5760" OR rule.id: "2502" OR rule.id: "5715")

This expression preserves the query documented in Section 4.4. The result list also included successful logins before the failed sequence. Those earlier successes were not counted as the later successful login. Opening full_log established which record occurred at 14:13:59 UTC.

Time and evidence fields

The search interval must include the relevant date, 11 October 2026. Dashboard times at UTC+3 must be distinguished from UTC source times. Windows comparisons used data.win.system.systemTime, data.win.system.eventID, data.win.system.eventRecordID and data.win.eventdata.targetUserName. Linux comparisons used full_log, data.dstuser, data.srcip and data.srcport. The queries select evidence for review; they do not implement a correlation rule.

Appendix 3 continued

Native verification commands

The commands below document the checks used during testing. SSH was initiated at a Windows PowerShell prompt. After authentication, commands entered at labuser@ubuntu-ssh-lab ran on Ubuntu through that session.

```
ssh labuser@192.168.56.102
```

```
whoami
hostname
date -Is
```

The Ubuntu outputs identified labuser and ubuntu-ssh-lab and supplied the displayed ISO-formatted system time. The native SSH journal was checked for the interval containing the final sequence:

```
sudo journalctl -u ssh.service \
--since "2026-10-11 14:05:00" \
--until "2026-10-11 14:15:00" \
--no-pager -o short-iso
```

This command reflects the tested Ubuntu session, whose time zone was UTC. It retrieved the failed passwords, PAM summary and later accepted password in the same source-log interval. The journal supports event ordering; corresponding Wazuh records support collection and rule classification. The journalctl options are described by systemd (n.d.).

On Windows, Event Viewer was used to inspect the Security log for event IDs 4624 and 4625. The target account, logon type, source time and event record ID were compared with the Wazuh details. The subject account was kept distinct from the account receiving or failing the logon session, following the field descriptions in Microsoft (2021a; 2026).

A complete deployment configuration, installed rule XML and effective audit/SSH settings were not exported during the recorded test series. Those files should accompany structured event exports in a future repeat of the experiment. Documentation examples, including those in the sources listed under References, are not treated as evidence that an identical configuration was deployed.